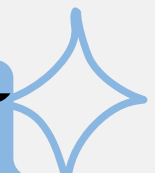




Full Stack Development Base on Reference Data Model



นำเสนอโดย นายสิทธิพงศ์ จำรัสฤทธิรงค์
อาจารย์ที่ปรึกษา รองศาสตราจารย์ ดร.วรา วราวิทย์

เส้นทางการเรียนรู้ ก่อนเริ่มงานจริง

01

ศึกษา Full Stack
Cloud Native
(Docker,
Postgres, PHP,
React)

02

เรียนรู้ Data Model
จาก Data Model
Resource Book

03

ออกแบบ Database
ด้วย Visual
Paradigm

04

สร้าง CRUD App
จัดการ Person ด้วย
Database จาก
หนังสือ

เส้นทางการเรียนรู้ ก่อนเริ่มงานจริง

05

สร้าง CRUD App จัดการ
Person ด้วย Database
จาก Visual Paradigm

06

สร้าง CRUD App จัดการ
Person ด้วย MVC
(React เป็น View)

07

สร้าง CRUD App
จัดการ Geo Data
(Laravel, Artisan)

08

ใช้ Pandas ETL(extract
transform load) นำ Dataset
Geo จากกรมการปกครองเข้าสู่
app

เส้นทางการเรียนรู้ ก่อนเริ่มงานจริง

09

พัฒนา Dynamic
Dropdown
(Country, Province,
District)

10

ปรับปรุง Laravel App
(Response Time: 7s
→ 1s)

11

สร้าง CRUD App
จัดการ Person-
Organization ด้วย
Slim PHP

12

พัฒนา CRUD App
เพิ่ม Role,
Relationship,
Communication
Event

เส้นทางการเรียนรู้ ก่อนเริ่มงานจริง

13

สร้าง CRUD App
เดิม เพิ่ม JWT
Authentication
ด้วย FastAPI

14

ปรับ UX/UI ตามคำ
แนะนำจาก
ผู้ช่วยศาสตราจารย์
ภาวิศร์ ณ รังษี
ครูสาขาออกแบบ

15

ปรับปรุง UI ตามข้อเสนอ
แนะจากผู้ใช้ Gen Z

16

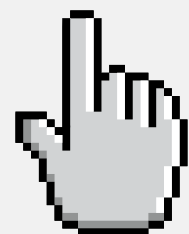
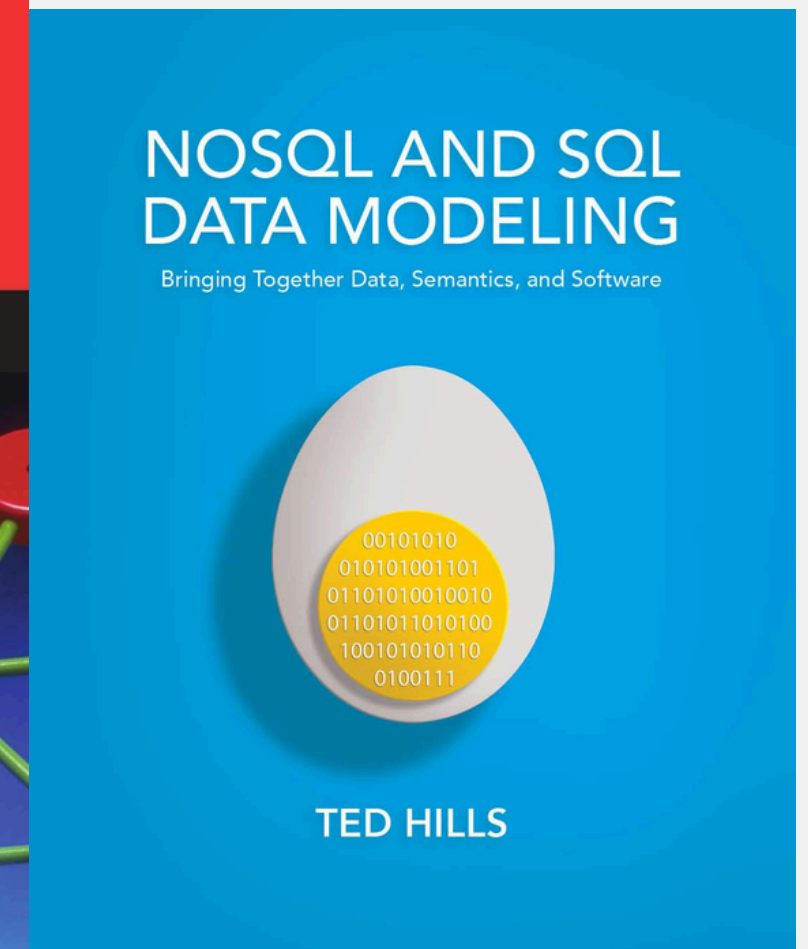
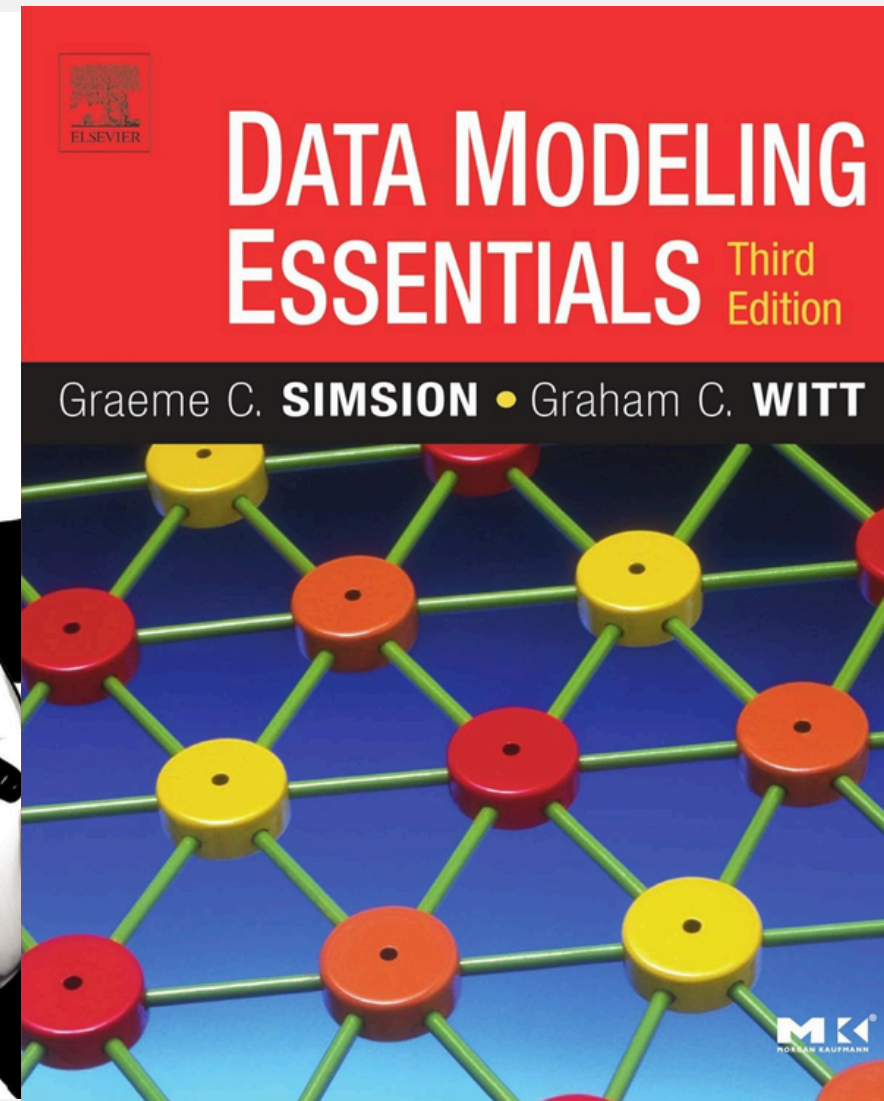
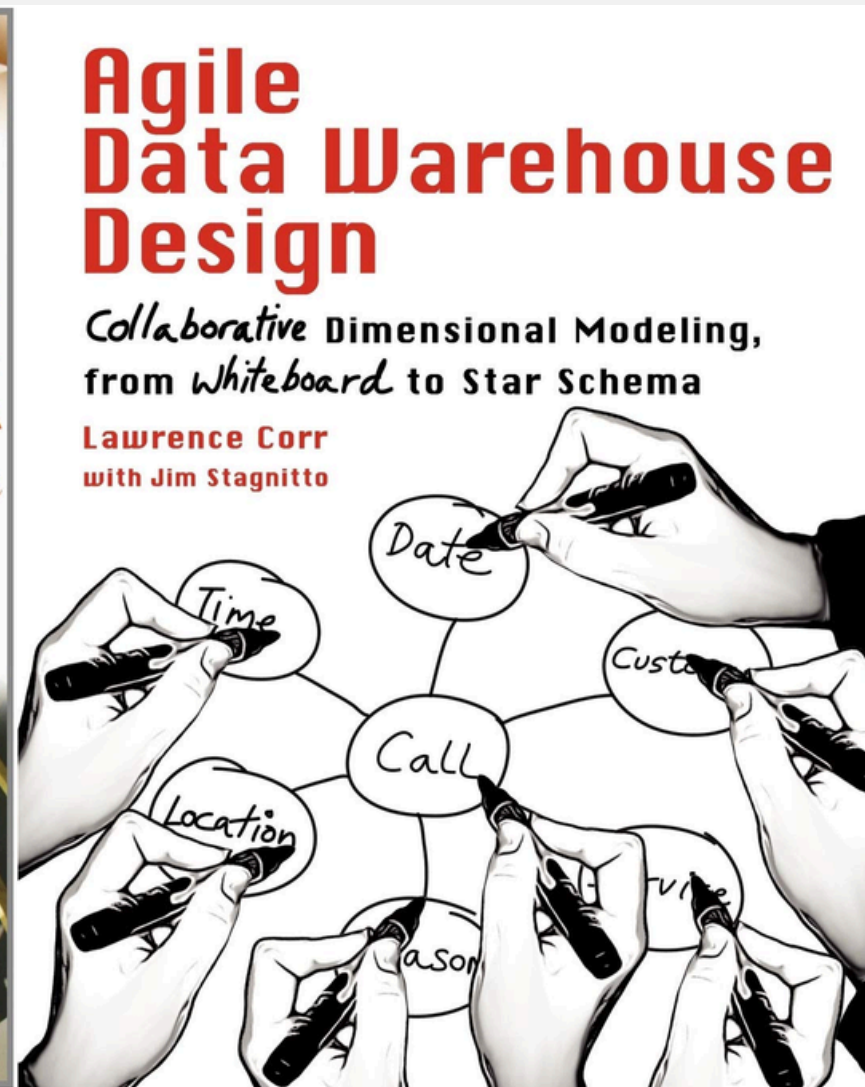
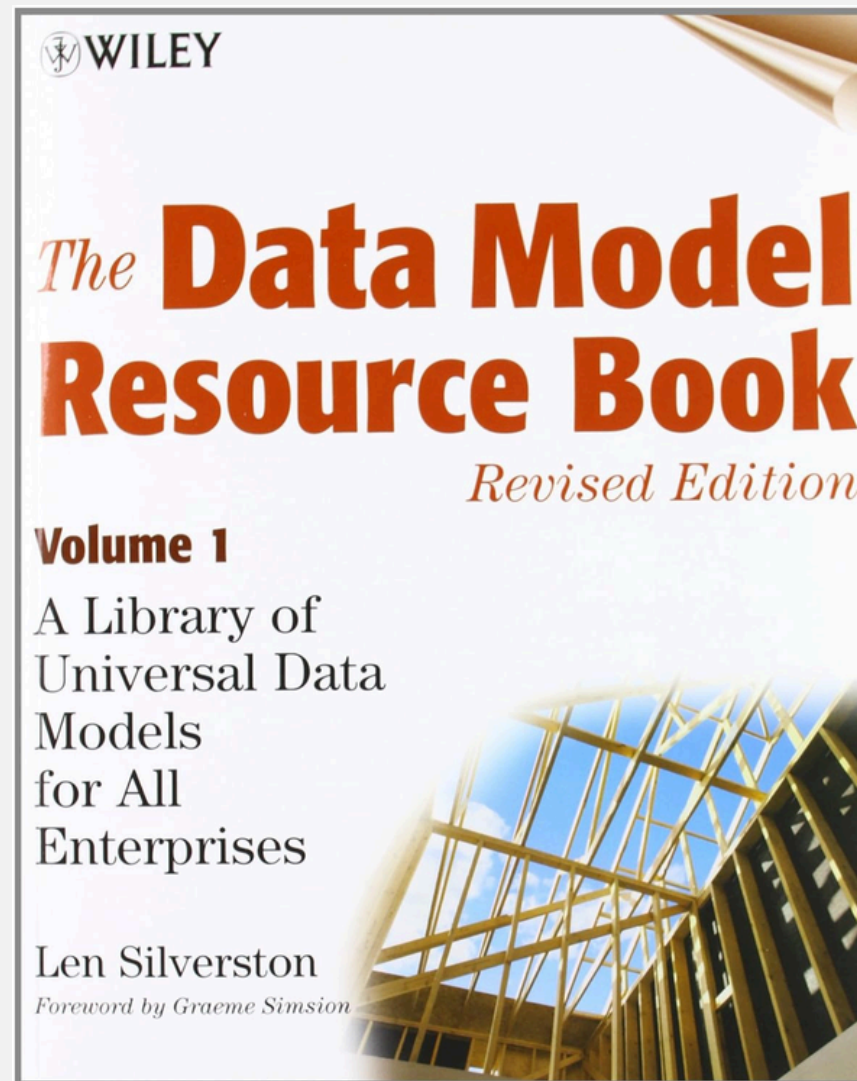
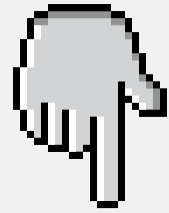
สร้าง Role-Based CRUD
App (6 บทบาท)

เส้นทางการเรียนรู้ ก่อนเริ่มงานจริง

◆
17

ออกแบบ Diagram ประกอบซอฟต์แวร์ (Activity, ERD, Sequence, State, Use Case)

Book About Data Model





Data Model

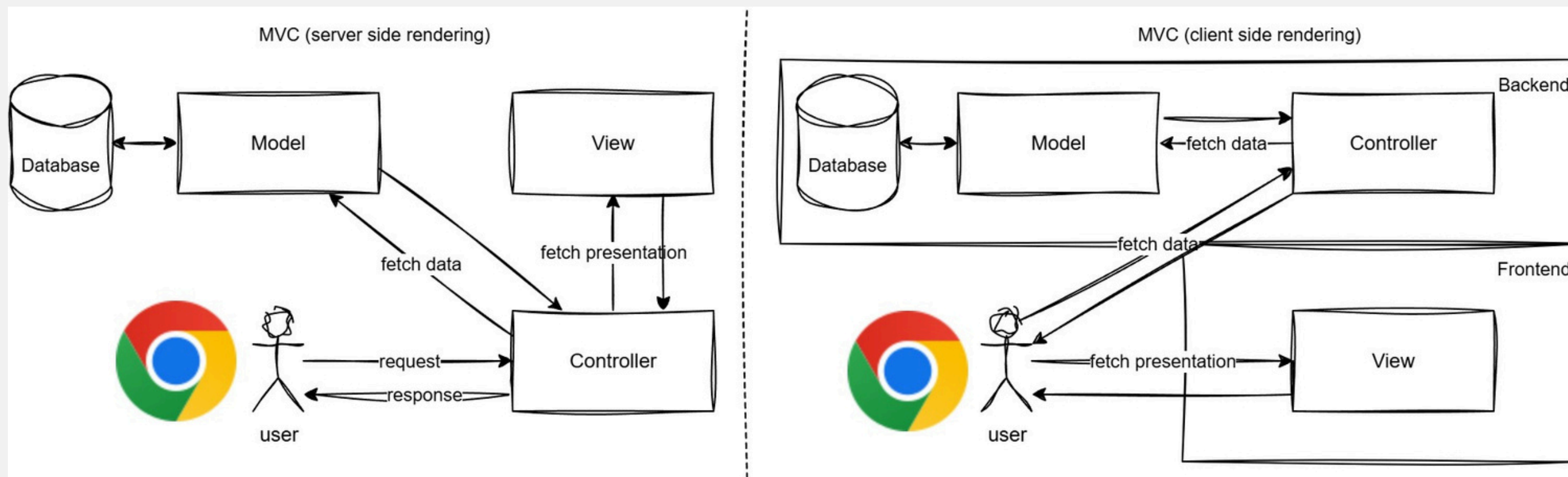
- Data คือข้อมูล
- Model คือรูปแบบ แบบจำลอง
- Data Model คือการสร้างแบบจำลองของข้อมูลในระบบ
- เมื่อ Developer ในทีมเห็นโครงสร้างชัดเจนก็จะเข้าใจข้อมูลในระบบมากขึ้น



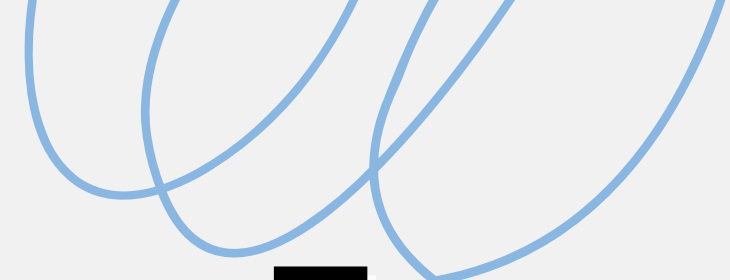
Evolution of Web

- Web 1.0 เว็บไซต์ static เน้นอ่านข้อมูลอย่างเดียว เหมือนหนังสือออนไลน์
- Web 2.0 เว็บไซต์ dynamic ผู้ใช้โต้ตอบและสร้างเนื้อหา เช่น social media
- ใช้ AJAX ทำให้หน้าเว็บอัปเดตแบบเรียลไทม์ เช่น Facebook, YouTube
- เริ่มจาก SSR สร้างหน้าเว็บที่เซิร์ฟเวอร์ ส่ง HTML ทั้งหน้า
- เปลี่ยนเป็น CSR ประมวลผลฝั่งผู้ใช้ ส่งเฉพาะ JSON

MVC แบบ SSR และ CSR



- MVC แบบ SSR ส่ง request ไป controller สร้างหน้าเว็บจาก server
- Controller ใน SSR จัดการ logic ดึง/แก้ไขข้อมูลจาก database แล้วส่ง View
- MVC แบบ CSR ใช้ frontend เช่น React เป็น View ส่ง request ไป backend
- Controller ใน CSR จัดการข้อมูล ส่งผลลัพธ์เป็น JSON ไปที่ frontend
- SSR เหมาะกับ monolithic ส่วน CSR เหมาะกับแอปสมัยใหม่



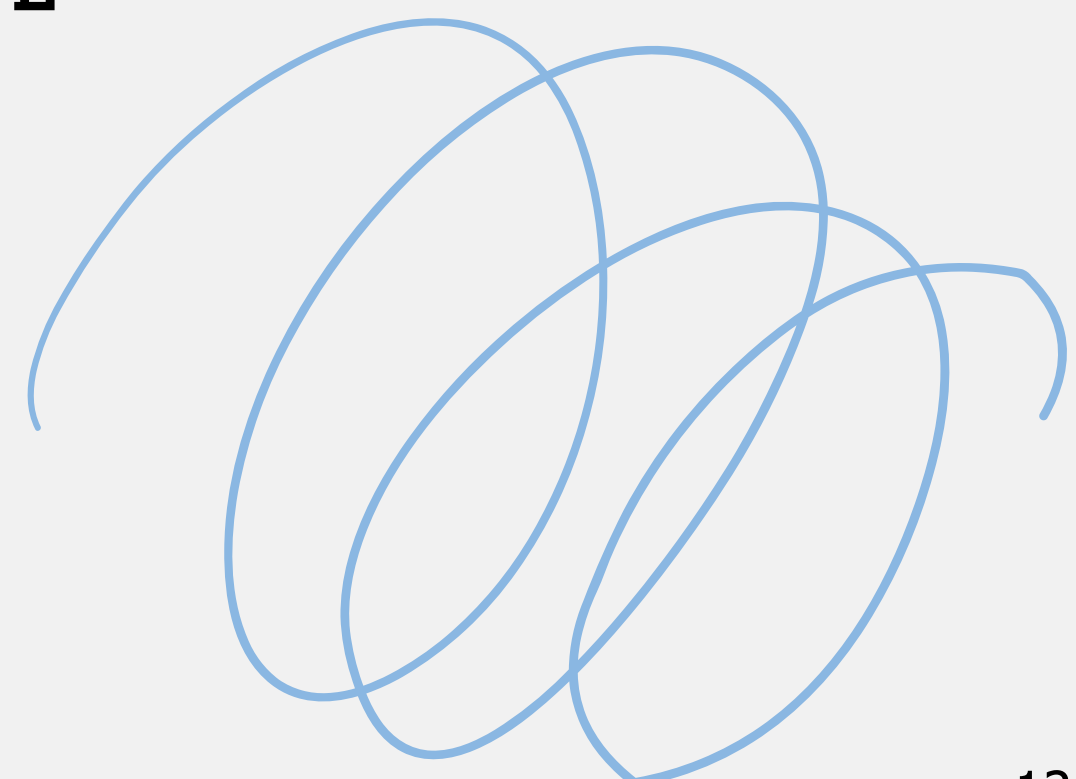
Framework

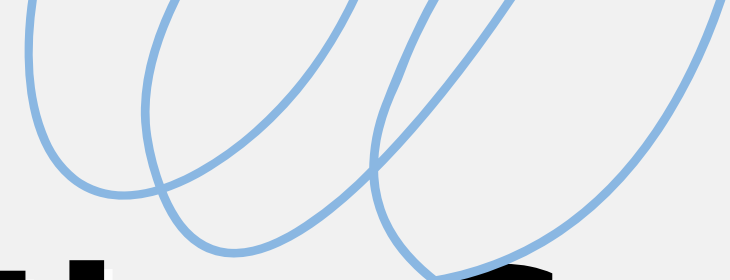
- Framework เป็นโครงสร้างกำหนดวิธีพัฒนาแอป มีกฎและ flow การทำงาน
 - ควบคุม flow เช่น MVC ใน Django จัดการ logic UI และ data
 - Library เป็นชุดโค้ดเรียกใช้ได้ตามต้องการ ไม่กำหนดโครงสร้าง
 - Framework บังคับโครงสร้าง (inversion of control) แต่ Library ให้อิสระ
 - ตัวอย่าง Framework เช่น Django React ส่วน Library เช่น jQuery Lodash
- 



Frontend และ Backend

- Frontend เป็นส่วนที่ผู้ใช้เห็นและโต้ตอบ เช่น UI ปุ่ม กล่องข้อความ Animation
- Backend เป็นสะพานเชื่อม Frontend กับ Database จัดการ logic และข้อมูล
- ร่วมกันสร้างประสบการณ์แอปที่สมบูรณ์และตอบสนองผู้ใช้
- ในเรื่อง MVC Backend คือ Model, Controller
- ในเรื่อง MVC Frontend คือ view





Application-Specific Framework

- เฟรมเวิร์กที่ออกแบบสำหรับงานเฉพาะรองรับเป้าหมาย

เฉพาะของแต่ละแอปพลิเคชัน

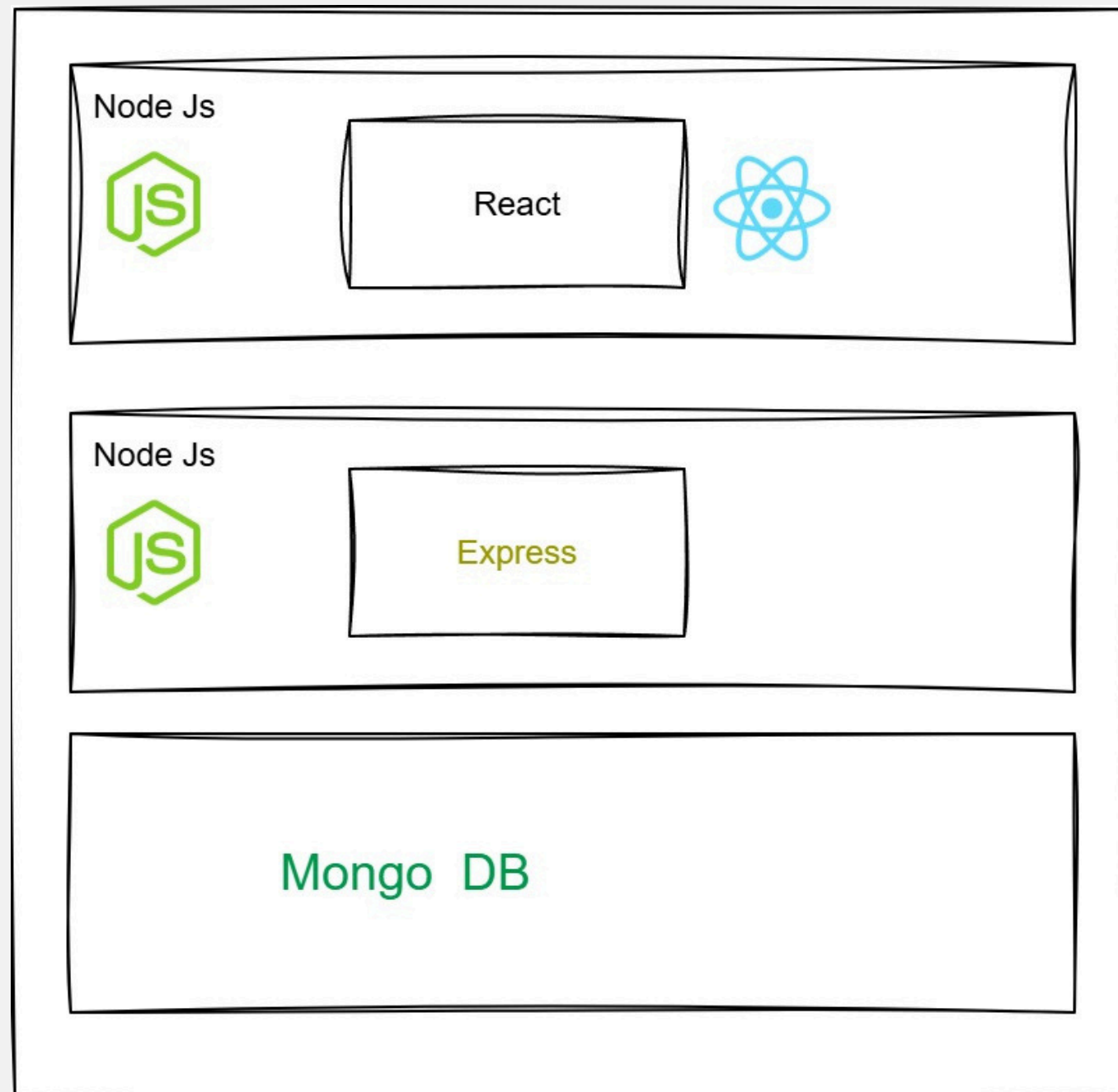
- ตัวอย่างเช่น
 - WordPress ใช้สร้างและจัดการเว็บไซต์เนื้อหา
 - Google Sites ใช้สร้างเว็บไซต์อย่างง่าย
 - Shopify ใช้พัฒนาร้านค้าออนไลน์
- 
- 



Frontend และ Backend Framework

- Frontend Framework สร้าง UI และการโต้ตอบแบบ dynamic
- ตัวอย่าง Frontend เช่น React, Vue.js, Angular ทำให้หน้าเว็บตอบสนองง่าย
- Backend Framework จัดการ logic ฝั่งเซิร์ฟเวอร์และ API
- ตัวอย่าง Backend เช่น Node.js, Django, Ruby on Rails
- ร่วมกันสร้างระบบที่เร็ว ปลอดภัย และใช้งานง่าย

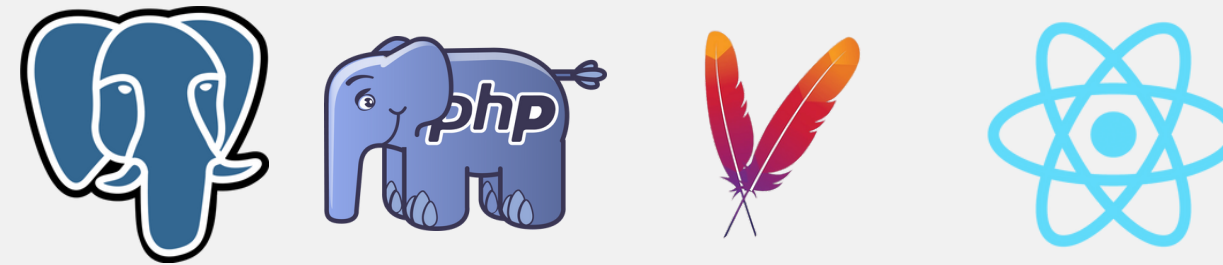
Framework Stack (Tech Stack)



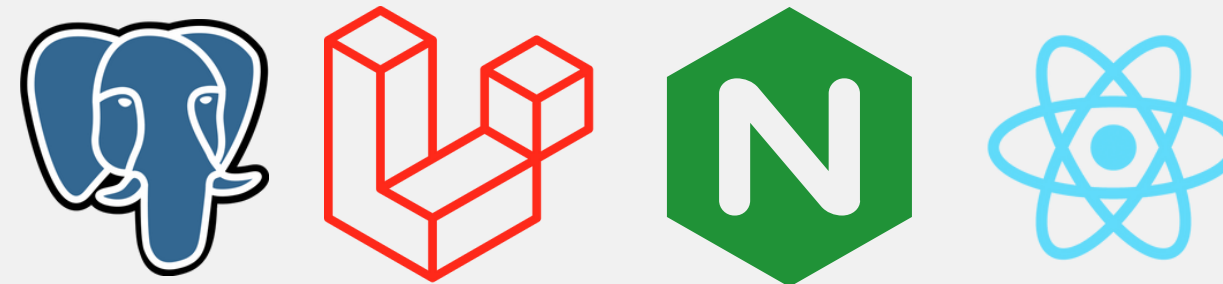
- ชุดเทคโนโลยีที่ใช้พัฒนาแอปหรือเว็บ
- รวม frontend, backend และฐานข้อมูล
- เช่น MERN Stack
- เลือก stack ให้เหมาะกับเป้าหมายโปรเจกต์

Tech Stack Timeline

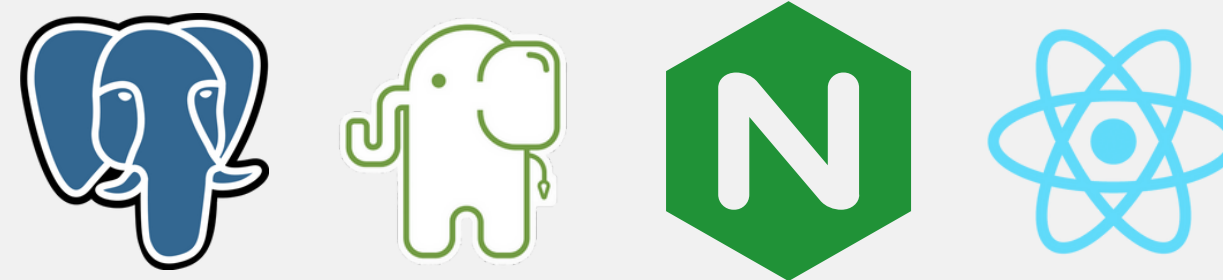
Postgres + Native PHP + React JS + Apache



Postgres + Laravel PHP + React TS + Nginx



Postgres + Slim PHP + React TS + Nginx

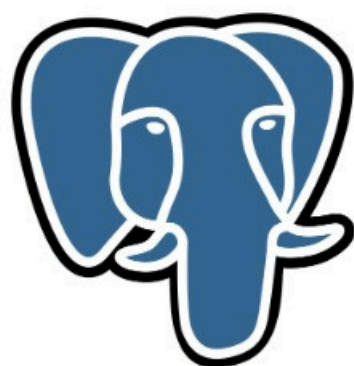


Postgres + FastAPI Python + React TS + Uvicorn

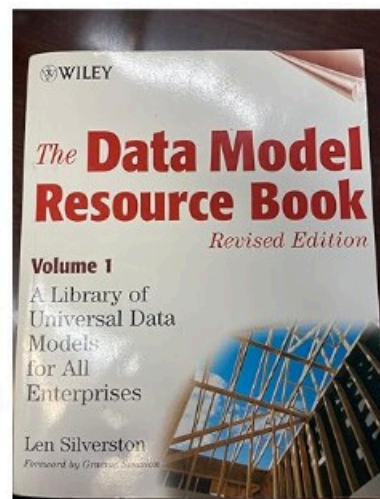


Docker Compose

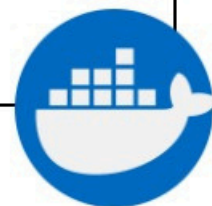
Database



PostgreSQL



Reference
Data Model



Backend (API)



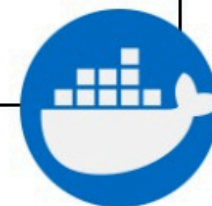
Fast API



Python



JWT OAuth2



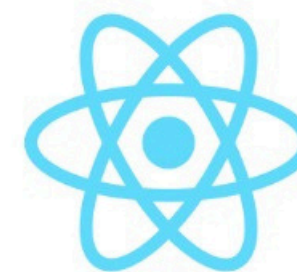
Frontend



Ts



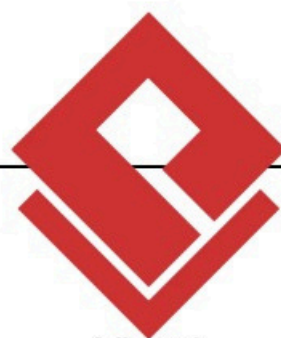
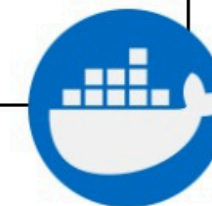
MUI



React



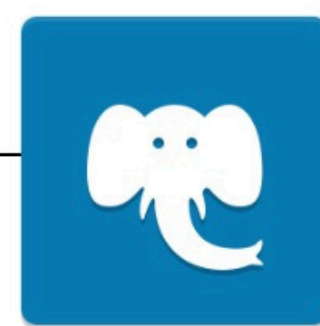
Vite



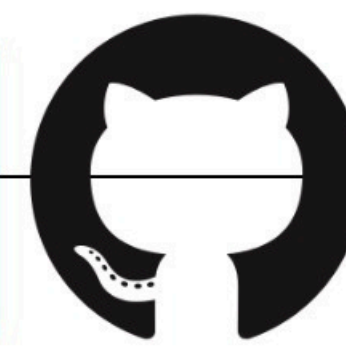
Visual
paradigm



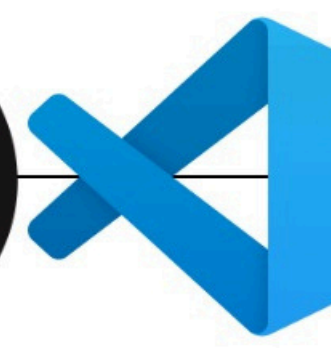
Dbeaver



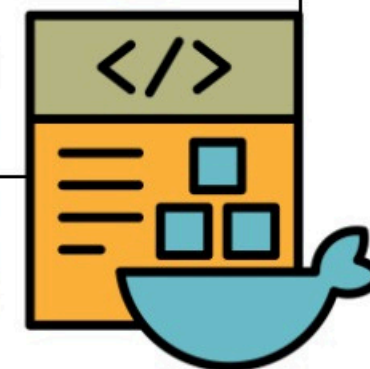
Pgadmin



Github



Vs Code Commu



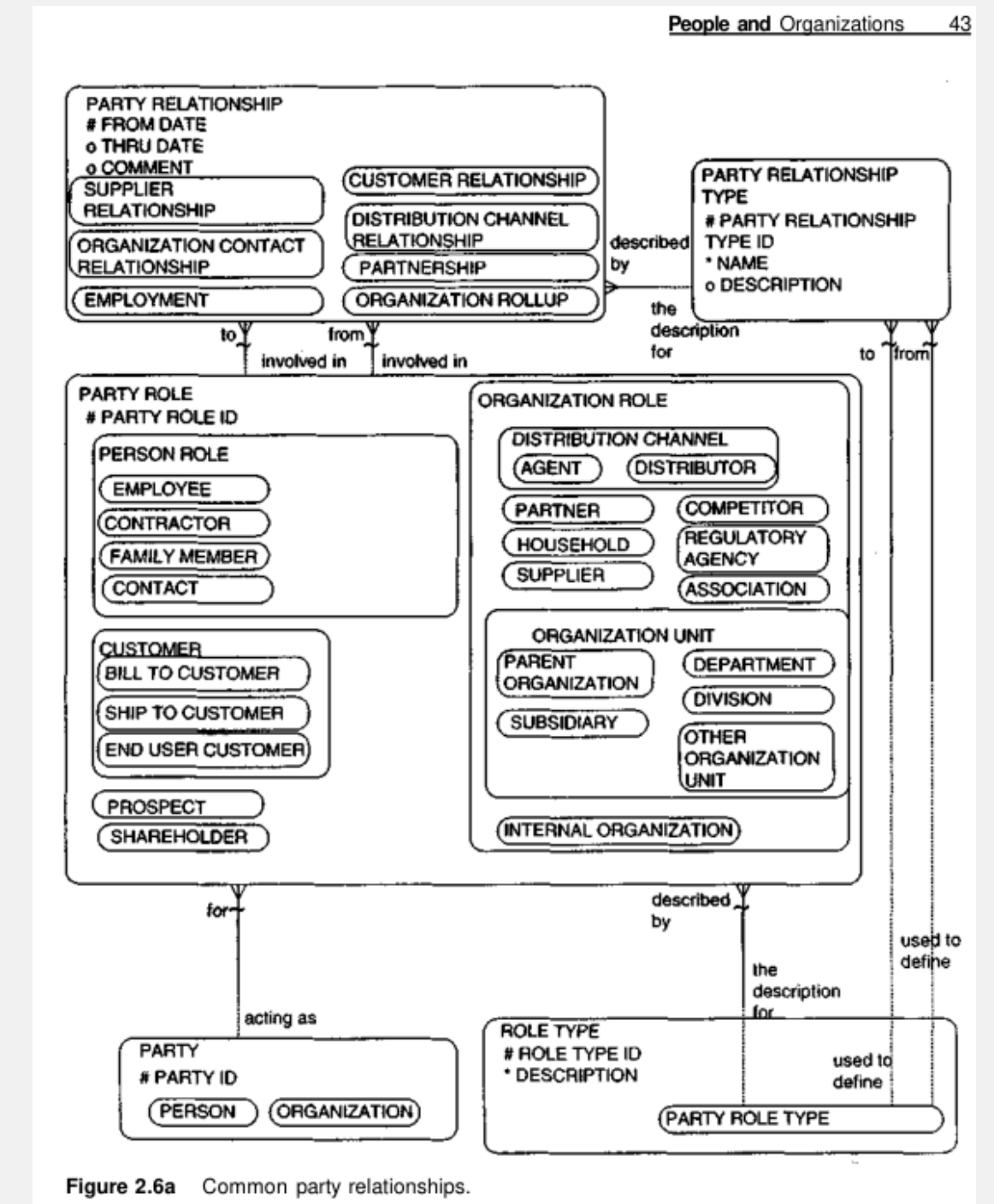
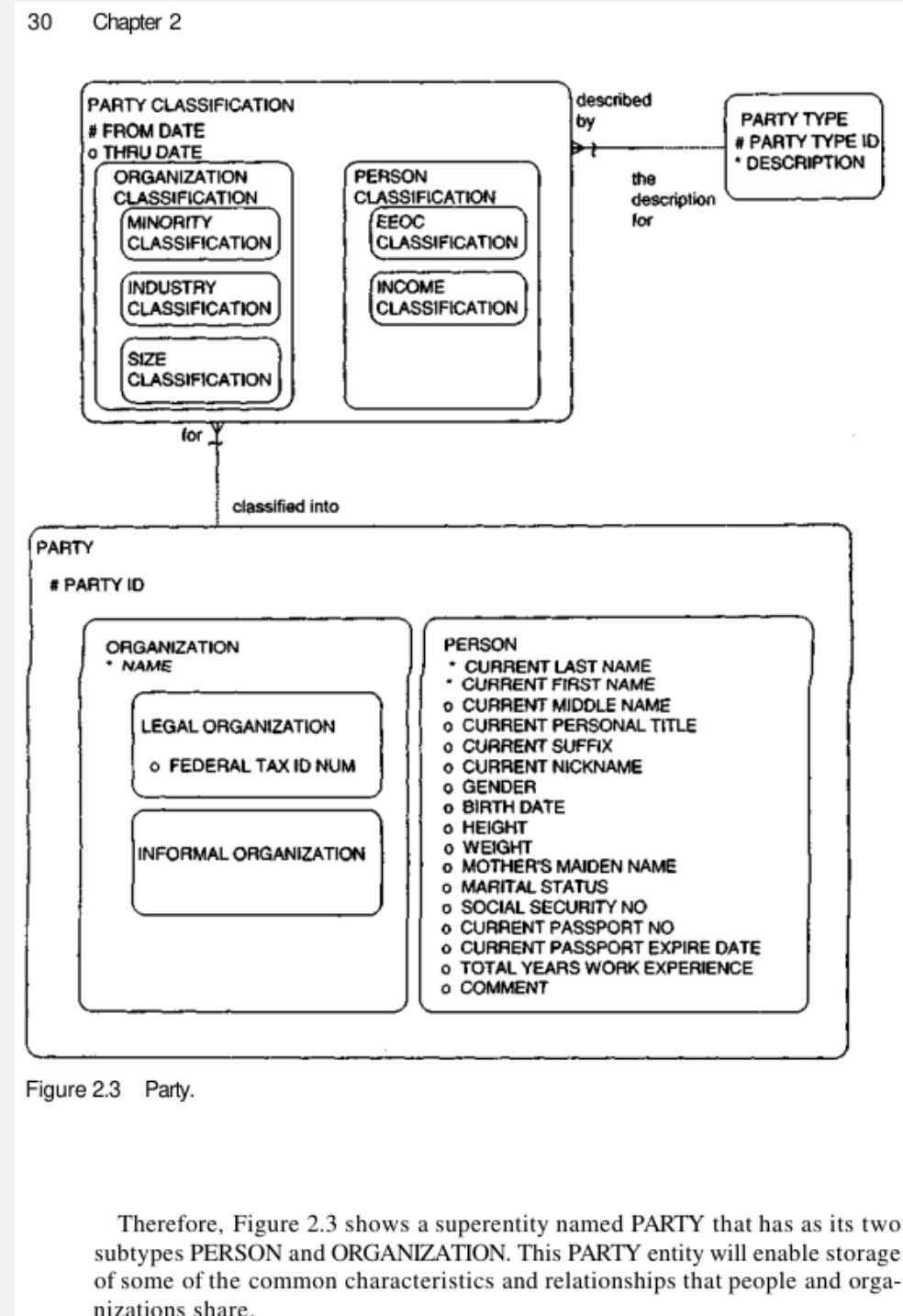
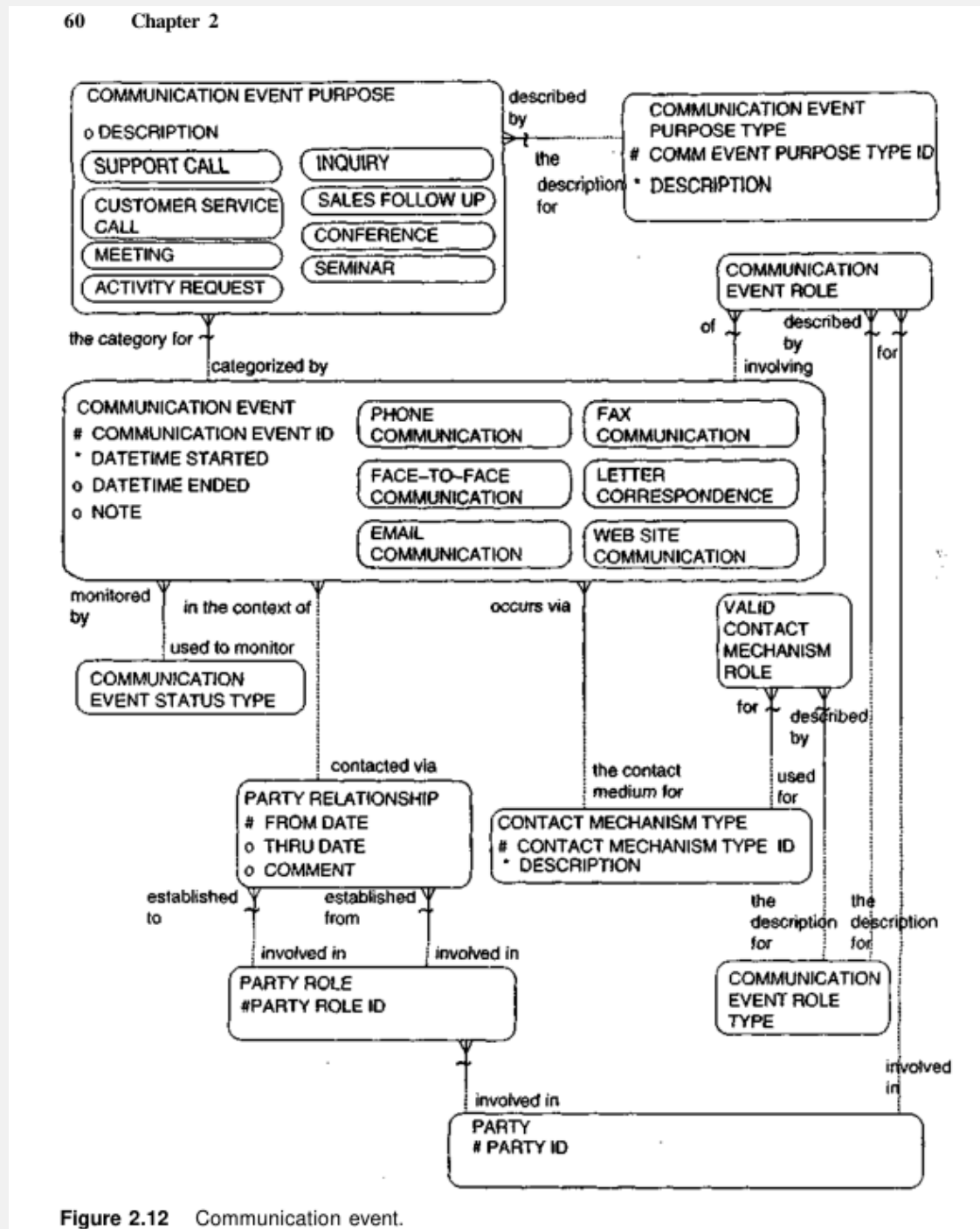


แนวคิดของโครงการ

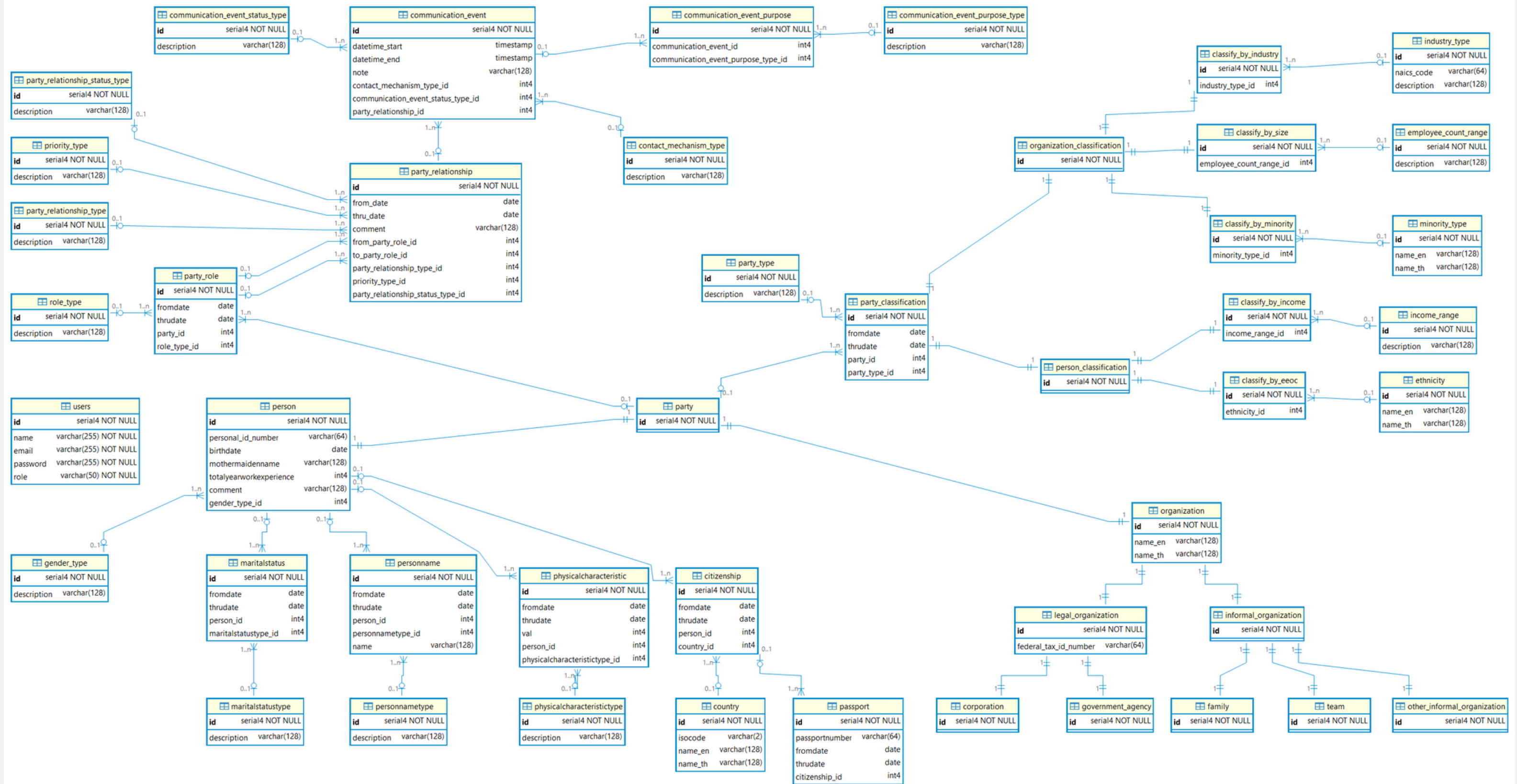


- ใช้ Party Model จากหนังสือสร้างแอป
 - CRUD Base Layer เช่น บุคคลและองค์กร ข้อมูลประเภทของสิ่งต่างๆ
 - CRUD Information Layer เช่น รายละเอียด บุคคล/องค์กร
 - CRUD Communication Layer การสื่อสารระหว่าง บุคคล/องค์กร
 - แอป Cloud Native พร้อม Authen ด้วย JWT และ OAuth 2.0
- 

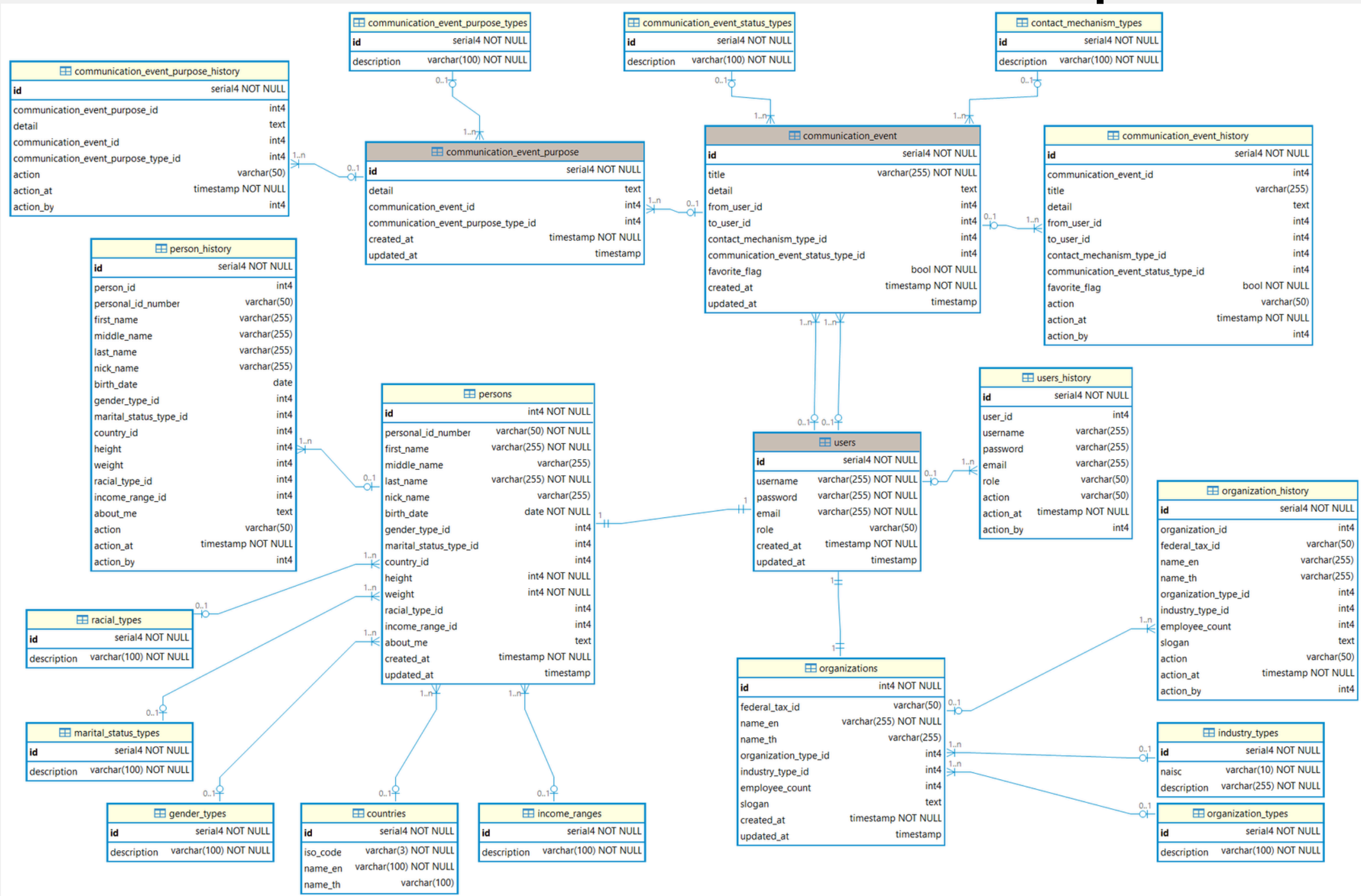
เลือก Party Model จากหนังสือมาต่อยอด

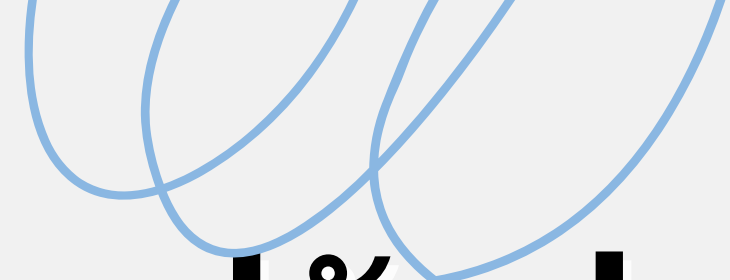


นำ Party Model มา implement



Database ฉบับปรับปรุง





การปรับปรุง Database

- ตัด subtype table ที่ไม่จำเป็นออกให้โครงสร้างกระชับ
 - เปลี่ยนการเก็บข้อมูลที่แปรตามเวลา จาก junction table (fromdate-thru date) เป็น history table
 - เพิ่มการเก็บประวัติว่าข้อมูลถูกทำอะไร เมื่อไหร่ โดยใคร
 - ลบ layer role และ relation เพื่อลดความซับซ้อน
 - ย้ายการ classification จาก subtype หลายชั้นมาเก็บใน person และ organization table
- 



มาตรฐาน ที่ใช้อ้างอิง

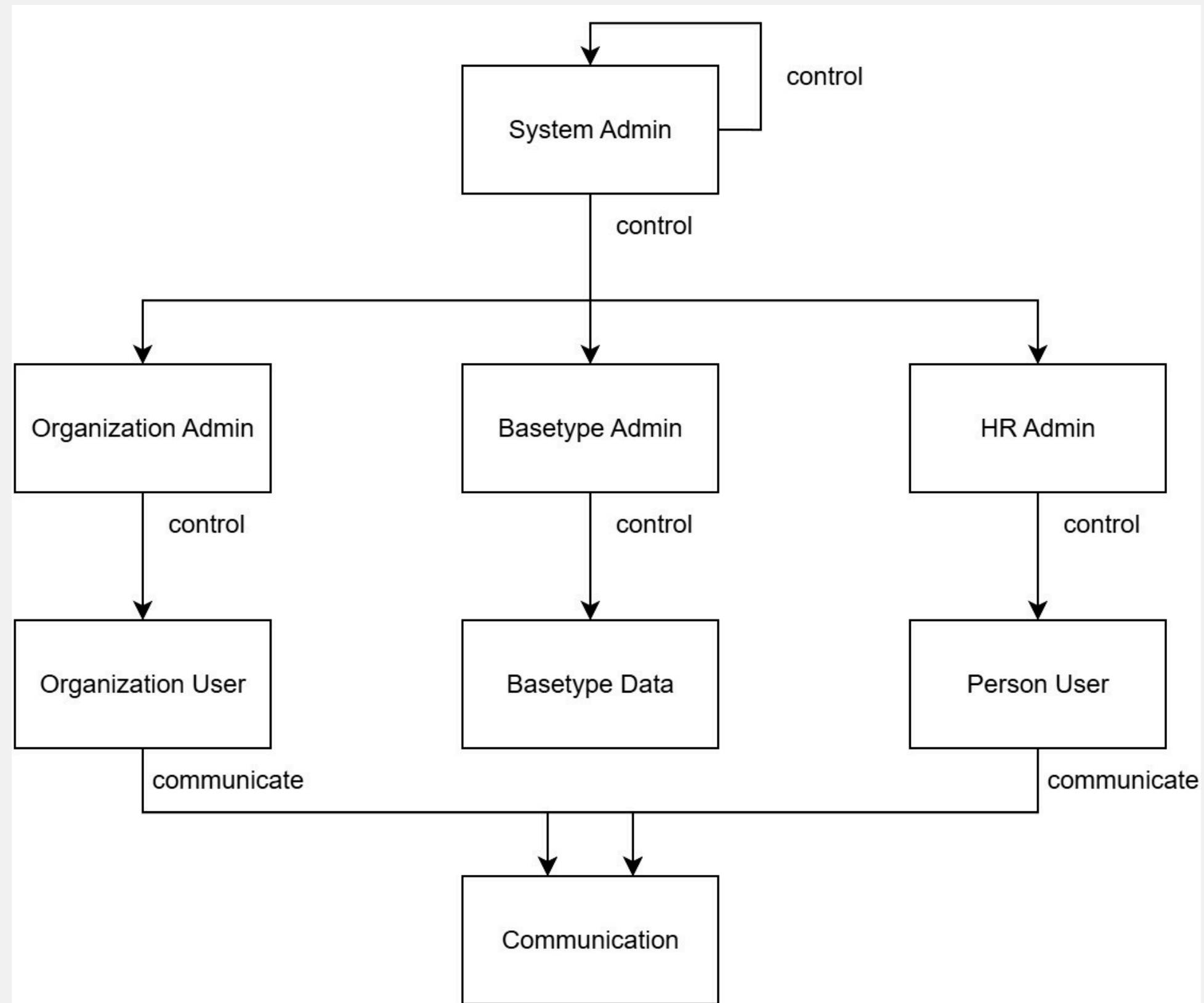
- Single Table Inheritance รวม persons และ organizations ในตาราง users ด้วย ID เดียว อ้างอิง Martin Fowler (Patterns of Enterprise Application Architecture)
- Denormalized Design รวมข้อมูลสำคัญ เช่น gender, marital status, industry ในตาราง persons และ organizations แทน junction table อ้างอิง Bill Karwin (SQL Antipatterns)
- Audit Logging บันทึกประวัติทุก entity เพื่อติดตามการเปลี่ยนแปลง อ้างอิง Snodgrass (Temporal Data Patterns)

จากทฤษฎีสู่การลงมือทำ



Functional Requirements

- Login เพื่อยืนยันตัวตนและความปลอดภัย
- System Admin สร้างและจัดการ account admin
- Organization Admin เพิ่มข้อมูล organization
- HR Admin เพิ่มข้อมูล person
- Organization และ Person User บันทึกการสื่อสาร
- รองรับฟีเจอร์ star และ filter ขาเข้า-ขาออก



สิทธิ์ของแต่ละ Role ในแอปพลิเคชัน

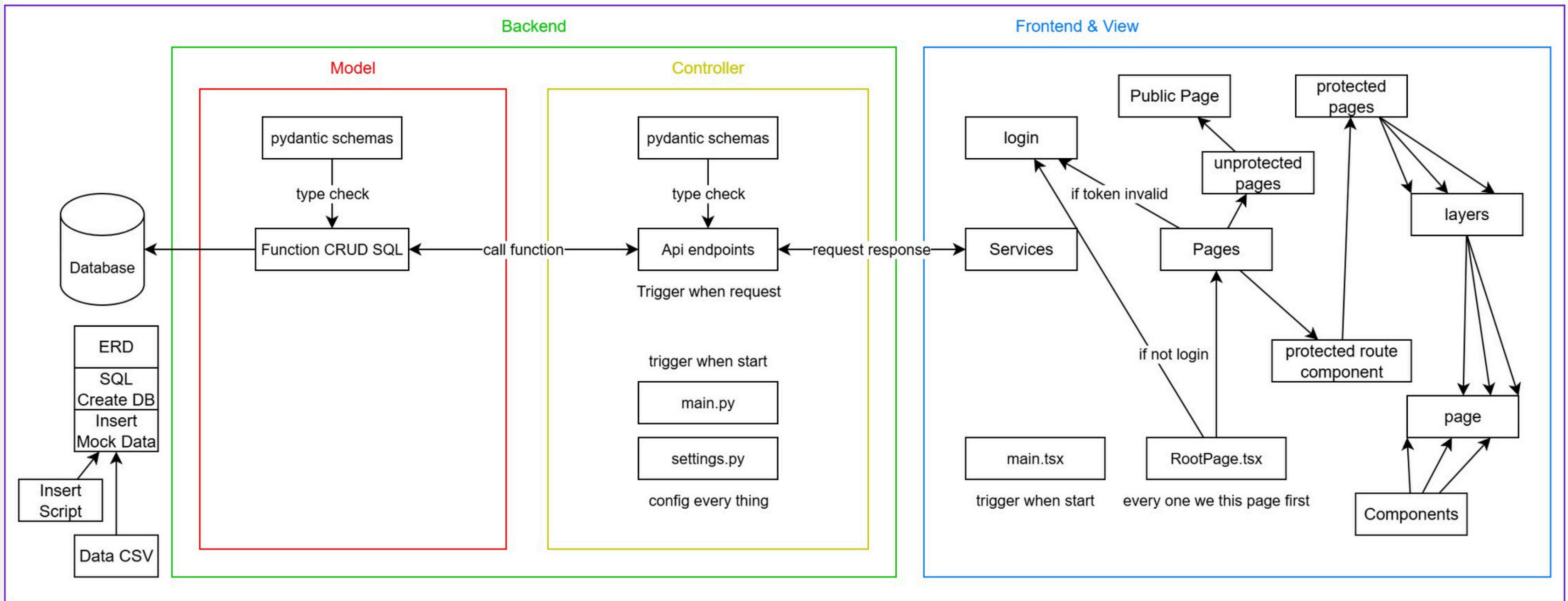


นำแนวคิดมาทำ Usecase Diagram

- มีทั้งหมด 6 role ในแอป
- System Admin จัดการ account Admin
- Basetype Admin จัดการข้อมูลพื้นฐาน เช่น country, gender_type
- Organization Admin ดูแล user ประเภท organization
- HR Admin ดูแล user ประเภท person
- Organization User ใช้แอปบันทึกการสื่อสาร
- Person User ใช้แอปบันทึกการสื่อสาร

ส่วนประกอบที่สำคัญของ App

Docker compose



Container Base

docker desktop

PERSONAL

Search

Ctrl+K

?

2

S

—

×

Ask Gordon BETA

Containers

Images

Volumes

Builds

Models BETA

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers

Give feedback

View all your running containers and applications.

Learn more

Container CPU usage

No containers are running.

Container memory usage

No containers are running.

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Actions
☐	party_fullstack_docker5	-	-	-	N/A	N/A	N/A	
☐	frontend-1	50fd23ffea6f	party_fullst	5173:5173	N/A	N/A	N/A	
☐	db-1	65856ab5d5ad	postgres:1	5432:5432	N/A	N/A	N/A	
☐	backend-1	8aea297ad936	party_fullst	8080:8080	N/A	N/A	N/A	

Showing 4 items

Engine running

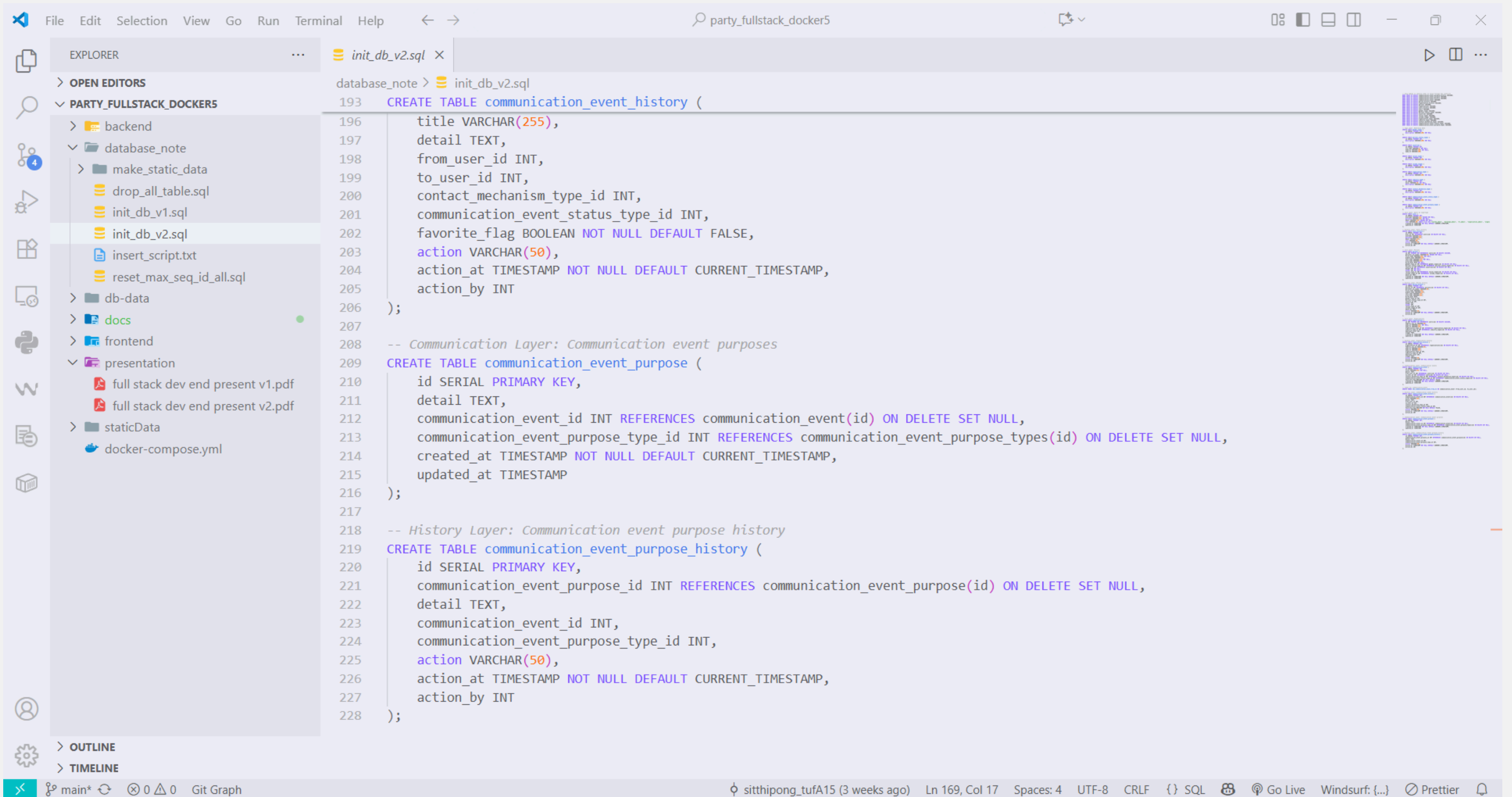
RAM 1.71 GB CPU 28.64% Disk: 50.48 GB used (limit 1006.85 GB)

Terminal

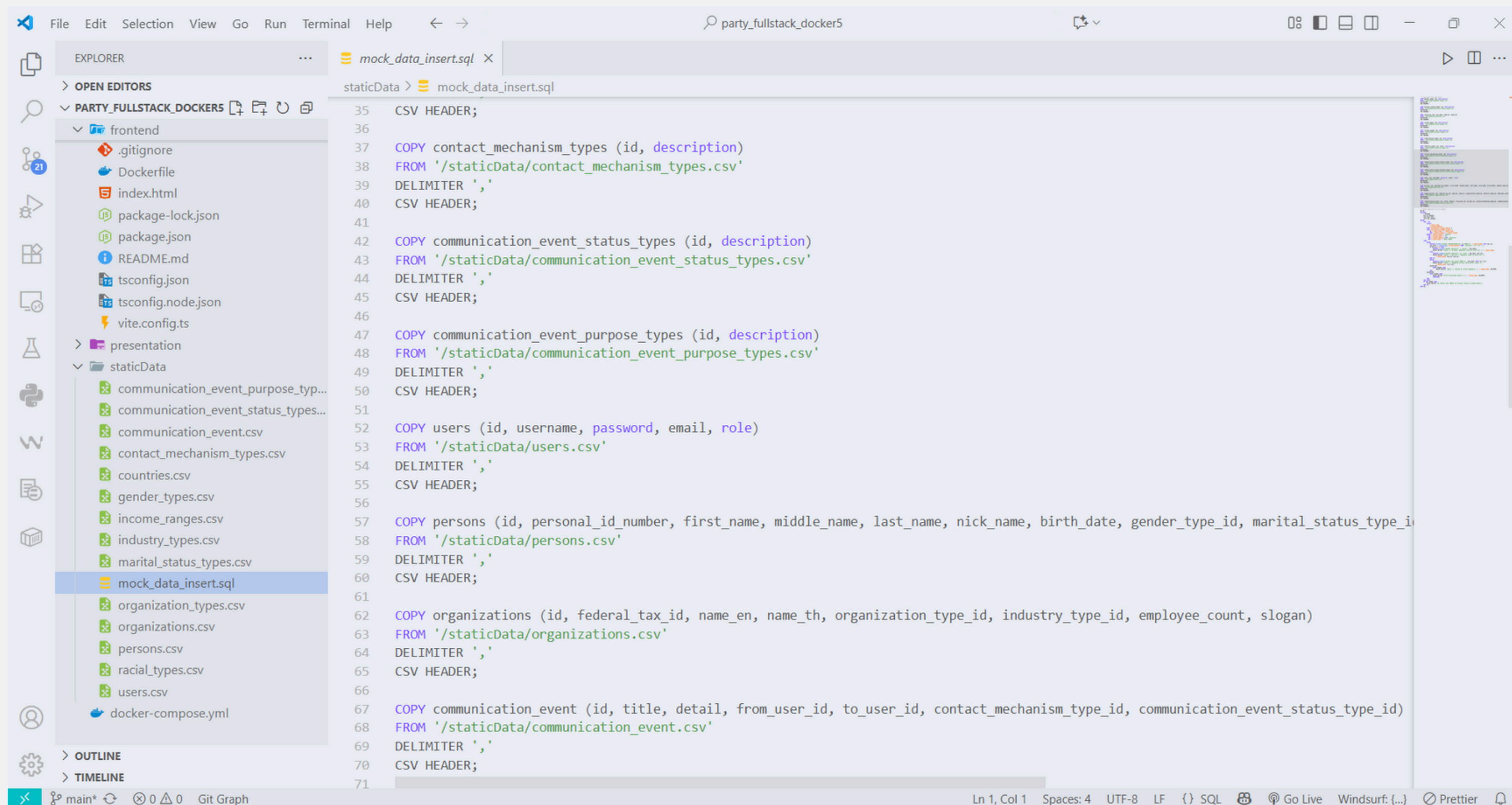
New version available

Compose และ Container ของ Project ที่ทำ

Database

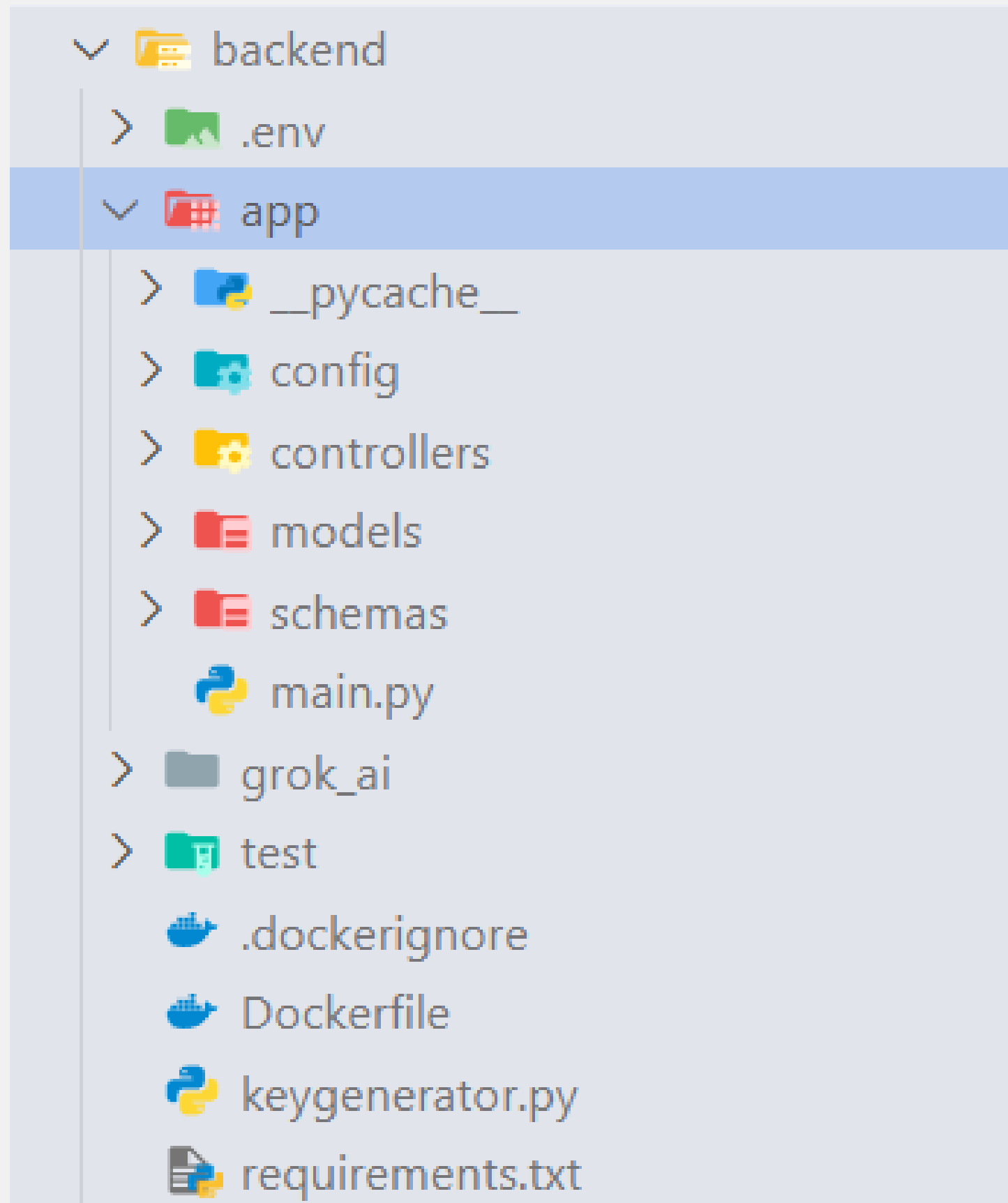


SQL ที่ใช้ในการสร้าง Database



Insert Mock Data

Backend



- แบ่ง backend เป็น 4 ส่วนหลัก
- Schemas ตรวจสอบ datatype ของ payload
- Models จัดการการเชื่อมต่อกับ database
- Controllers ควบคุมและจัดการ request
- main.py รวมทุก API endpoint
- ไม่มี View เพราะ Frontend ทำหน้าที่เป็น View

EXPLORER

> OPEN EDITORS

▼ PARTY_FULLSTACK_DOCKER5

▼ backend

▼ app

▼ schemas

▼ __pycache__

communication_event_history.py

communication_event_purpose_...

communication_event_status_ty...

communication_event.py

contact_mechanism_types.py

countries.py

gender_type.py

income_ranges.py

industry_types.py

marital_status_types.py

organization_history.py

organization_types.py

organization.py

person_history.py

person.py

racial_types.py

user.py

users_history.py

main.py

grok_ai

test

.dockerignore

Dockerfile

keygenerator.py

user.py

backend > app > schemas > user.py > UserUpdate

```
1 from pydantic import BaseModel, EmailStr
2 from typing import Optional
3 from datetime import datetime
4
5 # Schema สำหรับสร้างผู้ใช้
6 class UserCreate(BaseModel):
7     username: str
8     email: EmailStr
9     password: str
10    role: Optional[str] = None
11
12 # Schema สำหรับอัปเดตผู้ใช้
13 class UserUpdate(BaseModel):
14     username: Optional[str] = None
15     email: Optional[EmailStr] = None
16     password: Optional[str] = None
17     role: Optional[str] = None
18
19 # Schema สำหรับแสดงผลผู้ใช้
20 class UserOut(BaseModel):
21     id: int
22     username: str
23     email: EmailStr
24     role: Optional[str] = None
25     created_at: datetime
26     updated_at: Optional[datetime] = None
27
28 class Config:
29     from_attributes = True
30
31 # Schema สำหรับล็อกอิน
32 class UserLogin(BaseModel):
33     email: EmailStr
34     password: str
```

EXPLORER

> OPEN EDITORS

▼ PARTY_FULLSTACK_DOCKER5

▼ backend

▼ app

▼ schemas

income_ranges.py

industry_types.py

marital_status_types.py

organization_history.py

organization_types.py

organization.py

person_history.py

person.py

racial_types.py

user.py

users_history.py

main.py

grok_ai

test

.dockerignore

Dockerfile

keygenerator.py

requirements.txt

database_note

db-data

docs

frontend

presentation

users_history.py

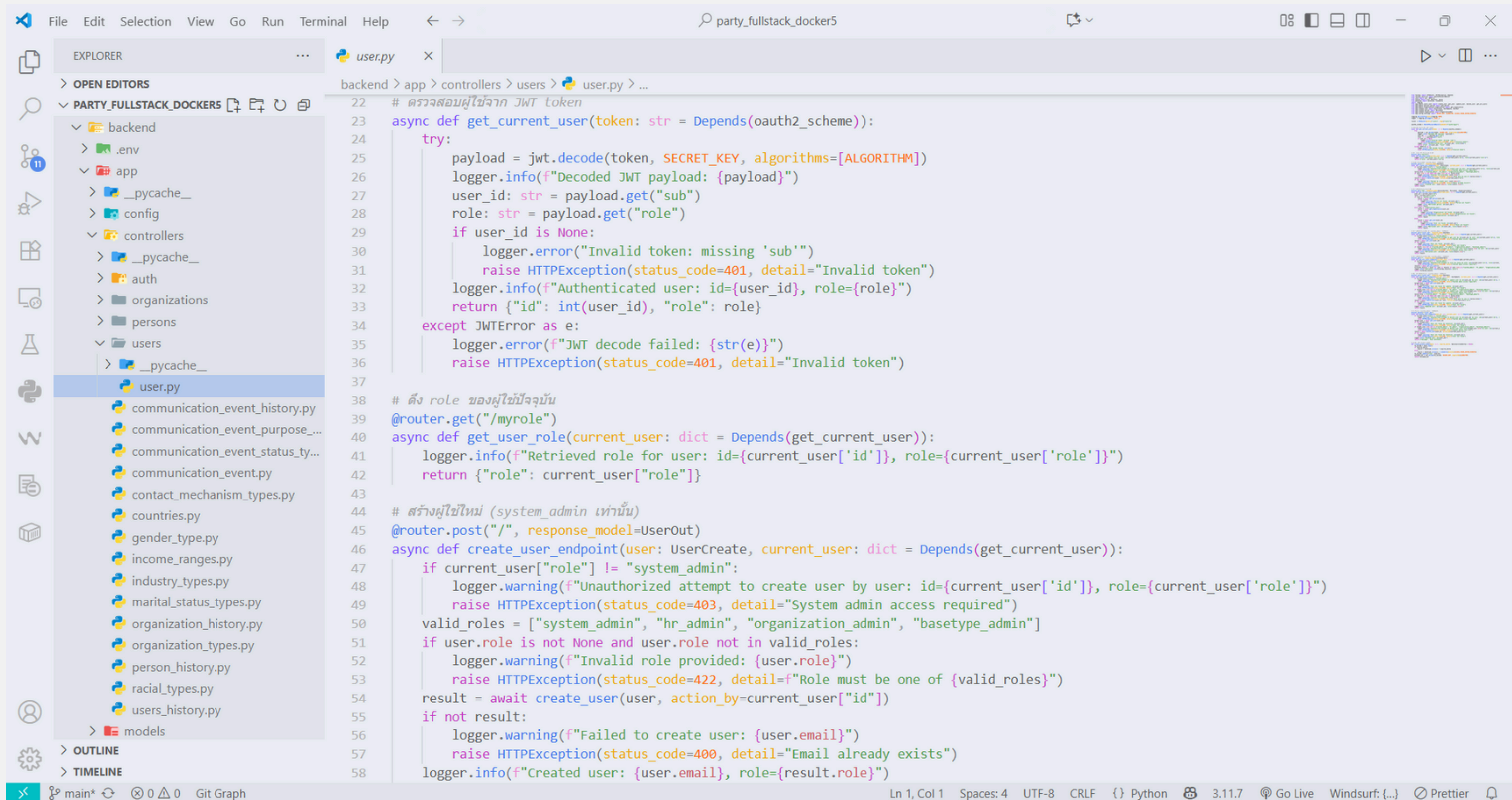
backend > app > schemas > users_history.py > UsersHistoryOut

```
1 from pydantic import BaseModel
2 from typing import Optional
3 from datetime import datetime
4
5 # Schema สำหรับแสดงผล users_history
6 class UsersHistoryOut(BaseModel):
7     id: int
8     user_id: Optional[int]
9     username: Optional[str]
10    password: Optional[str]
11    email: Optional[str]
12    role: Optional[str]
13    action: Optional[str]
14    action_at: datetime
15    action_by: Optional[int]
16
17 class Config:
18     from_attributes = True
```

schema ของ endpoint user, user_history

```
38 # สร้างผู้ใช้ใหม่
39 async def create_user(user: UserCreate, action_by: Optional[int]) -> Optional[UserOut]:
40     async with database.transaction():
41         try:
42             existing_user = await get_user_by_email(user.email)
43             if existing_user:
44                 logger.warning(f"Attempt to create user with existing email: {user.email}")
45                 return None
46             hashed_password = bcrypt.hashpw(user.password.encode('utf-8'), BCRYPT_SALT.encode('utf-8')).decode('utf-8')
47             now = datetime.utcnow()
48             query = """
49             INSERT INTO users (username, email, password, role, created_at)
50             VALUES (:username, :email, :password, :role, :created_at)
51             RETURNING id, username, email, role, created_at, updated_at
52             """
53             values = {
54                 "username": user.username,
55                 "email": user.email,
56                 "password": hashed_password,
57                 "role": user.role or "hr_admin",
58                 "created_at": now
59             }
60             result = await database.fetch_one(query=query, values=values)
61             if result:
62                 await log_user_history(
63                     user_id=result["id"],
64                     username=user.username,
65                     email=user.email,
66                     password=hashed_password,
67                     role=user.role or "hr_admin",
68                     action="create",
69                     action_by=action_by
70                 )
71                 logger.info(f"Created user: {user.email}, role={user.role}")
72                 return UserOut(**result._mapping)
73             return None
74 except ValueError as e:
```

ตัวอย่าง Code Model (function CRUD SQL)



ตัวอย่าง Code Controller (API endpoint)


```

1 from dotenv import load_dotenv
2 from fastapi import FastAPI
3 from fastapi.middleware.cors import CORSMiddleware
4 from contextlib import asynccontextmanager
5 from app.config.database import database
6 from app.controllers.auth.auth import router as auth_router
7 from app.controllers.users.user import router as user_router
8 from app.controllers.persons.person import router as person_router
9 from app.controllers.organizations.organization import router as organization_router
10 from app.controllers.gender_type import router as gender_type_router
11 from app.controllers.marital_status_types import router as marital_status_type_router
12 from app.controllers.countries import router as country_router
13 from app.controllers.racial_types import router as racial_type_router
14 from app.controllers.income_ranges import router as income_range_router
15 from app.controllers.organization_types import router as organization_type_router
16 from app.controllers.industry_types import router as industry_type_router
17 from app.controllers.contact_mechanism_types import router as contact_mechanism_type_router
18 from app.controllers.communication_event_status_types import router as communication_event_status_type_router
19 from app.controllers.communication_event_purpose_types import router as communication_event_purpose_type_router
20 from app.controllers.users_history import router as users_history_router
21 from app.controllers.person_history import router as person_history_router
22 from app.controllers.organization_history import router as organization_history_router
23 from app.controllers.communication_event import router as communication_event_router
24 from app.controllers.communication_event_history import router as communication_event_history_router
25
26 load_dotenv()
27
28 # Define lifespan event handler for FastAPI application
29 @asynccontextmanager
30 async def lifespan(app: FastAPI):
31     await database.connect()
32     yield
33     await database.disconnect()

```

```

1 # Initialize FastAPI app with lifespan event handler
2 app = FastAPI(lifespan=lifespan)
3
4 # Configure CORS to allow all origins for development
5 app.add_middleware(
6     CORSMiddleware,
7     allow_origins=["*"],
8     allow_credentials=True,
9     allow_methods=["*"],
10    allow_headers=["*"],
11 )
12
13 app.include_router(auth_router)
14 app.include_router(user_router)
15 app.include_router(person_router)
16 app.include_router(organization_router)
17 app.include_router(gender_type_router)
18 app.include_router(marital_status_type_router)
19 app.include_router(country_router)
20 app.include_router(racial_type_router)
21 app.include_router(income_range_router)
22 app.include_router(organization_type_router)
23 app.include_router(industry_type_router)
24 app.include_router(contact_mechanism_type_router)
25 app.include_router(communication_event_status_type_router)
26 app.include_router(communication_event_purpose_type_router)
27 app.include_router(users_history_router)
28 app.include_router(person_history_router)
29 app.include_router(organization_history_router)
30 app.include_router(communication_event_router)
31 app.include_router(communication_event_history_router)
32
33 @app.get("/")
34 async def root():
35     return {"message": "FastAPI Backend", "github": "https://github.com/sssZz-TH/party_fullstack_docker5"}

```

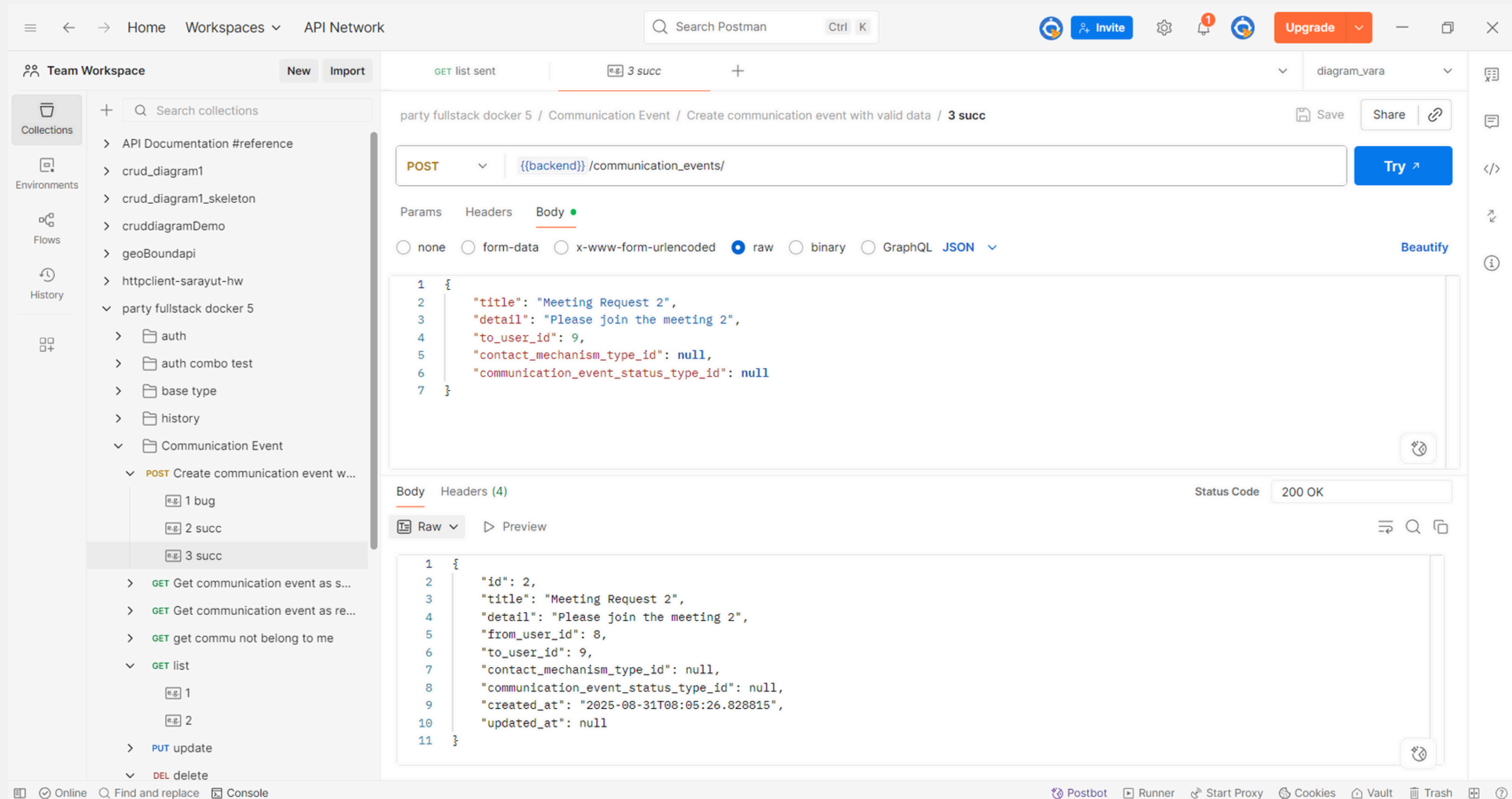
main.py รวมทุก API endpoint

The screenshot shows the Visual Studio Code interface with a project named 'party_fullstack_docker5'. The Explorer sidebar on the left shows the project structure, with the 'config' directory expanded and 'settings.py' selected. The main editor window displays the contents of 'settings.py'.

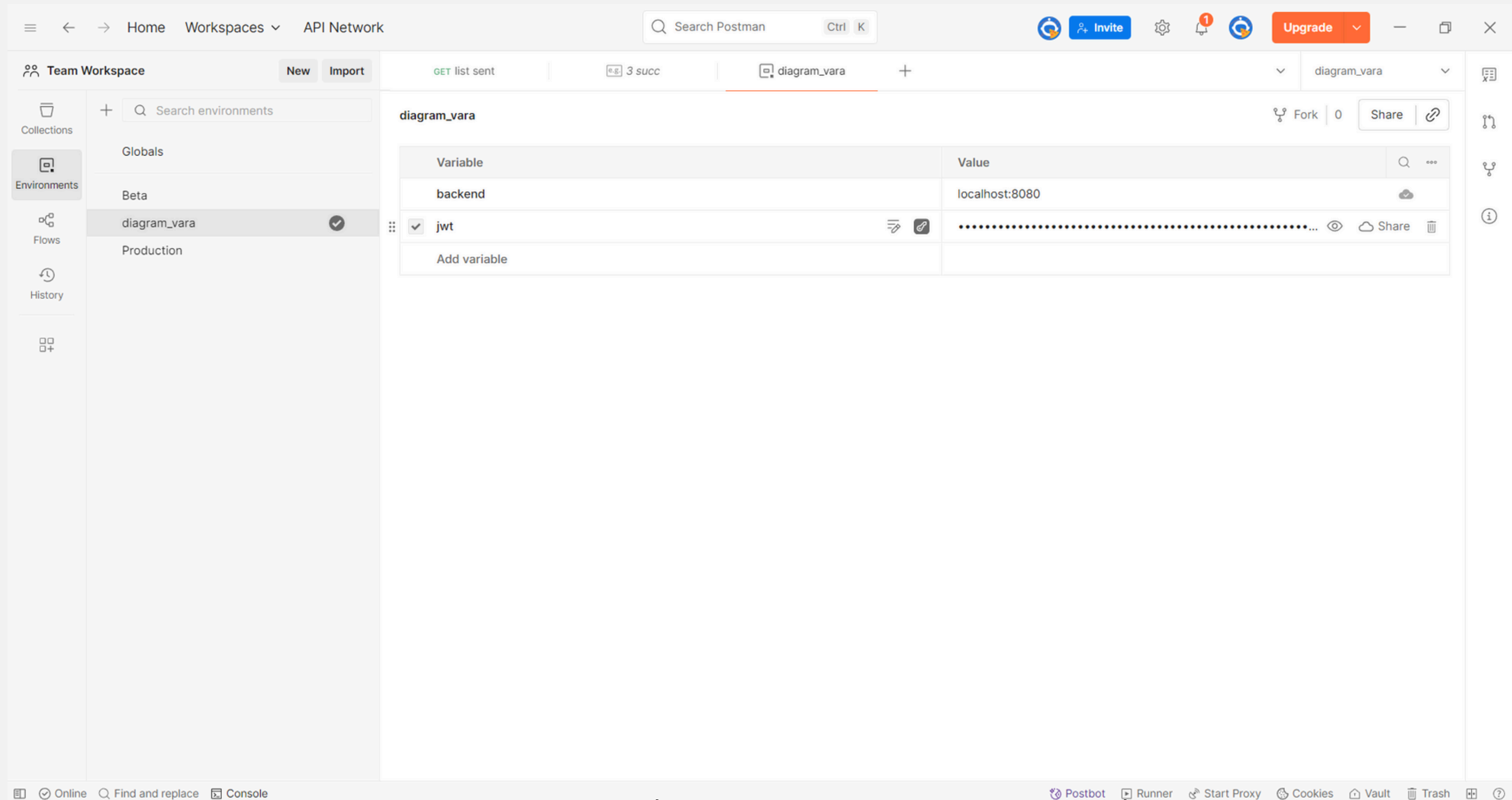
```
1 import re
2 import logging
3 from datetime import timedelta
4
5 logging.basicConfig(level=logging.DEBUG)
6 logger = logging.getLogger(__name__)
7
8 DATABASE_URL = "postgresql://spa:spa@db:5432/myapp"
9 API_HOST = "0.0.0.0"
10 API_PORT = 8080
11 SECRET_KEY = "8c2f7a9b3d6e1f0c4a8b2d5e7f9a1c3b6d8e0f2a4b7c9d1e3f5a8b0c2d4e6f"
12 ALGORITHM = "HS256"
13 BCRYPT_SALT = "$2b$12$zDZMoHsxUdSvpUNjEzsve"
14 ACCESS_TOKEN_EXPIRE_MINUTES = 1440 # 1 day in minutes
15 ALLOWED_ORIGINS = ["http://localhost:5173"]
16 ALLOWED_METHODS = ["GET", "POST", "PUT", "DELETE"]
17
18 logger.info(f"Loaded BCrypt_SALT: {BCRYPT_SALT}")
19 if not BCrypt_SALT:
20     raise ValueError("BCRYPT_SALT is not set")
21 if not re.match(r'^$2b$\d{2}\$[A-Za-z0-9./]{22}$', BCrypt_SALT):
22     raise ValueError(f"BCRYPT_SALT is invalid: {BCRYPT_SALT}. It must be in the format $2b$<cost>$<22-char-base64>")
```

การ deploy ในอนาคต สามารถ setting ทุกอย่างได้ในทีเดียว

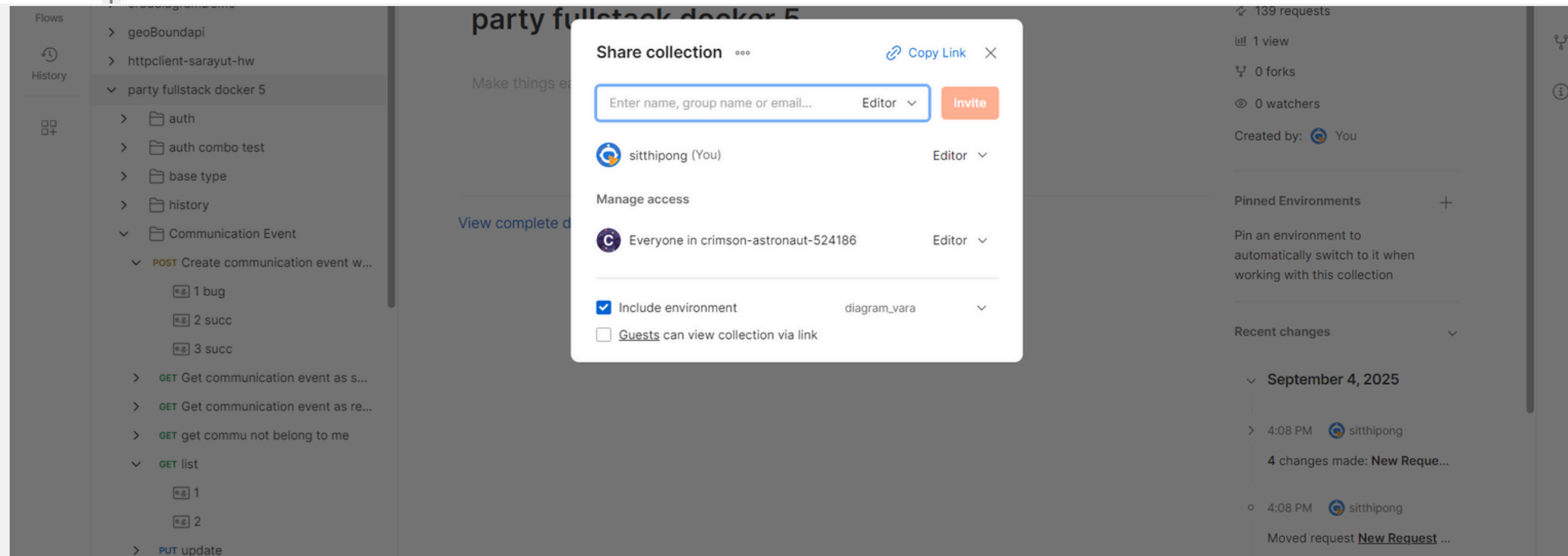
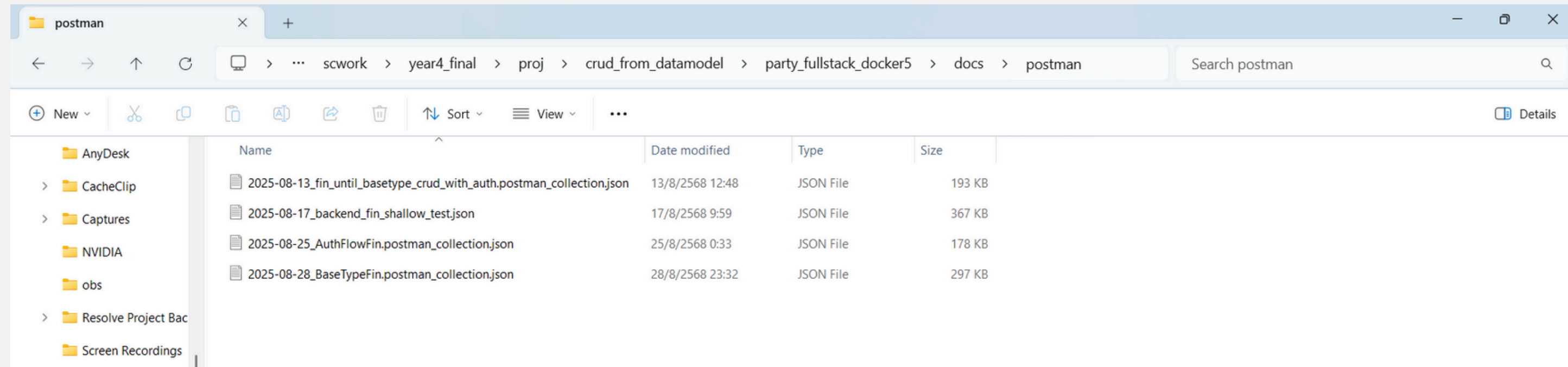
Backend Testing



บันทึกผลการ test ด้วย Postman



Environment เก็บ JWT เพื่อนำไปใช้ test service ที่มีการป้องกัน



export ผลการ test เป็น json ขึ้น github และ share link team

Team Workspace | New | Import | Runner | diagram_vara

Search Postman | Ctrl K

Search collections

party fullstack docker 5

- auth
- auth combo test
- base type
- history
- Communication Event
- GET root api
- API Documentation #reference
- crud_diagram1
- crud_diagram1_skeleton
- cruddiagramDemo
- geoBoundapi
- httpClient-sarayut-hw
- party model
- party_fullstack_docker4
- pokemonApi
- product_assoc
- REST API basics: CRUD, test & variable
- RESTful API Basics #blueprint
- sbd tester
- test_slim_fw

Run Sequence

Deselect All | Select All | Reset

- GET users history
- GET user history
- GET persons history
- GET person history
- GET organizations history
- GET organization history
- GET communication event history
- GET communication event 1 history
- POST Create communication event with valid data
- GET Get communication event as sender
- GET Get communication event as recipient
- GET get commu not belong to me
- GET list
- PUT update
- DEL delete
- POST New Request
- GET list sent
- 1 [x] GET root api

Functional | Performance

Test how your APIs perform under load

Simulate real-world traffic from your local machine and observe the performance of your APIs. Learn more about [performance testing](#)

Set up your performance test

Load profile | Virtual users | Test duration

Ramp up | 20 | 5 mins

20 VUs

0 | 5 mins

5 virtual users run for 1:20 minutes, ramp up to 20 for 1:10 minutes, then maintain 20 for 2:30 minutes, each executing all requests sequentially.

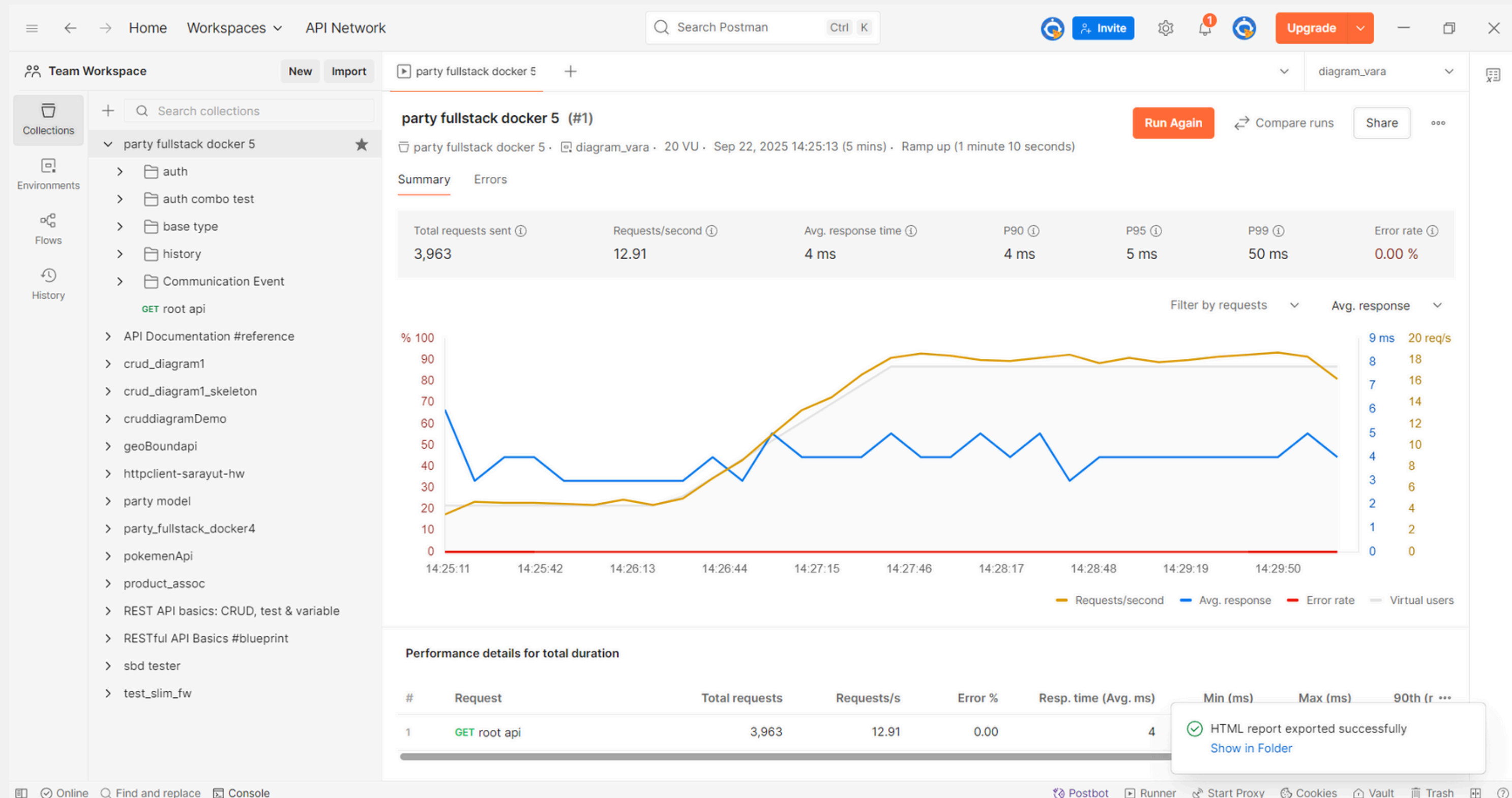
Initial load | 5

Data file | Select file

Pass test if

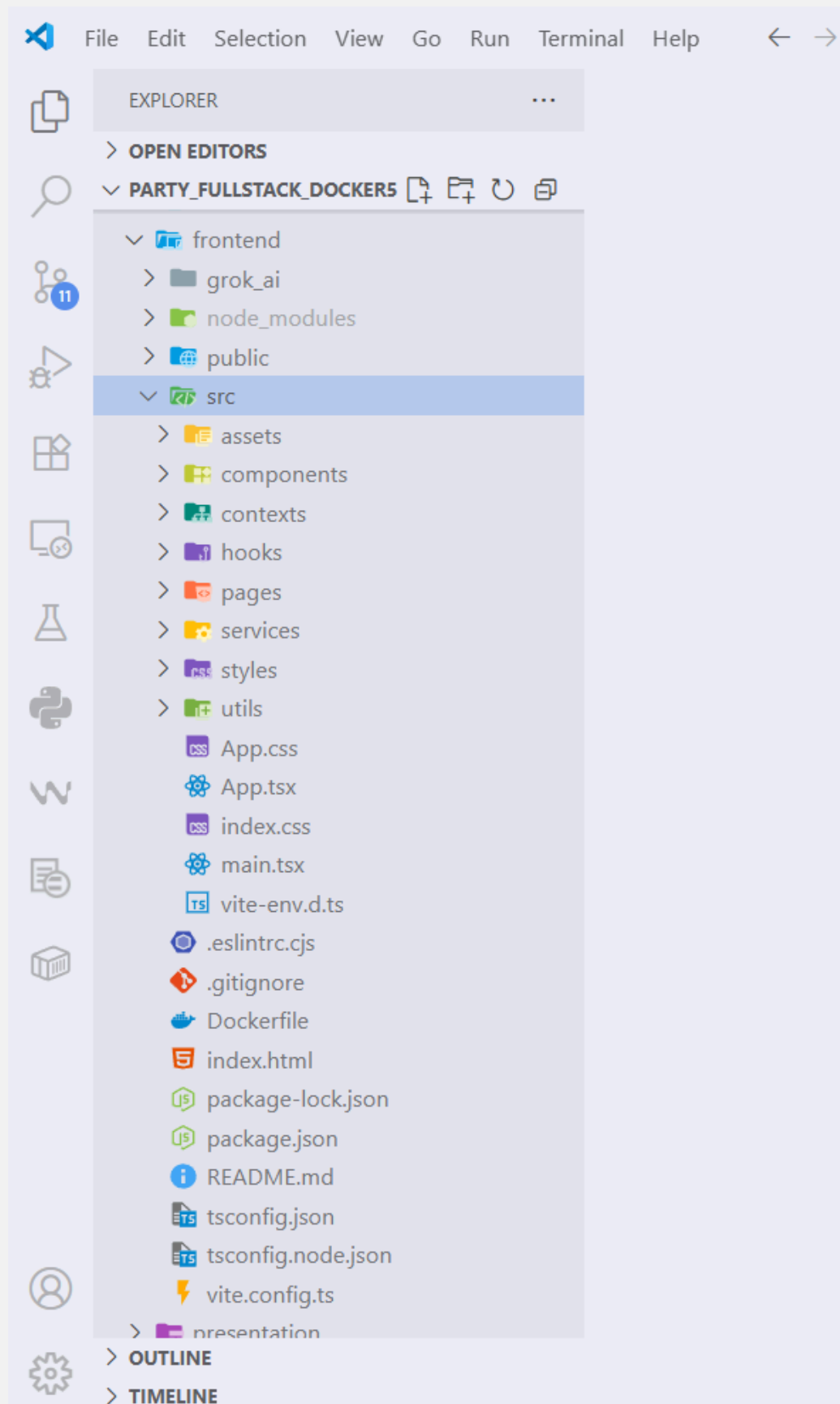
Postbot | Runner | Start Proxy | Cookies | Vault | Trash

ตั้งค่าก่อน Load Test



ทำ Load Test

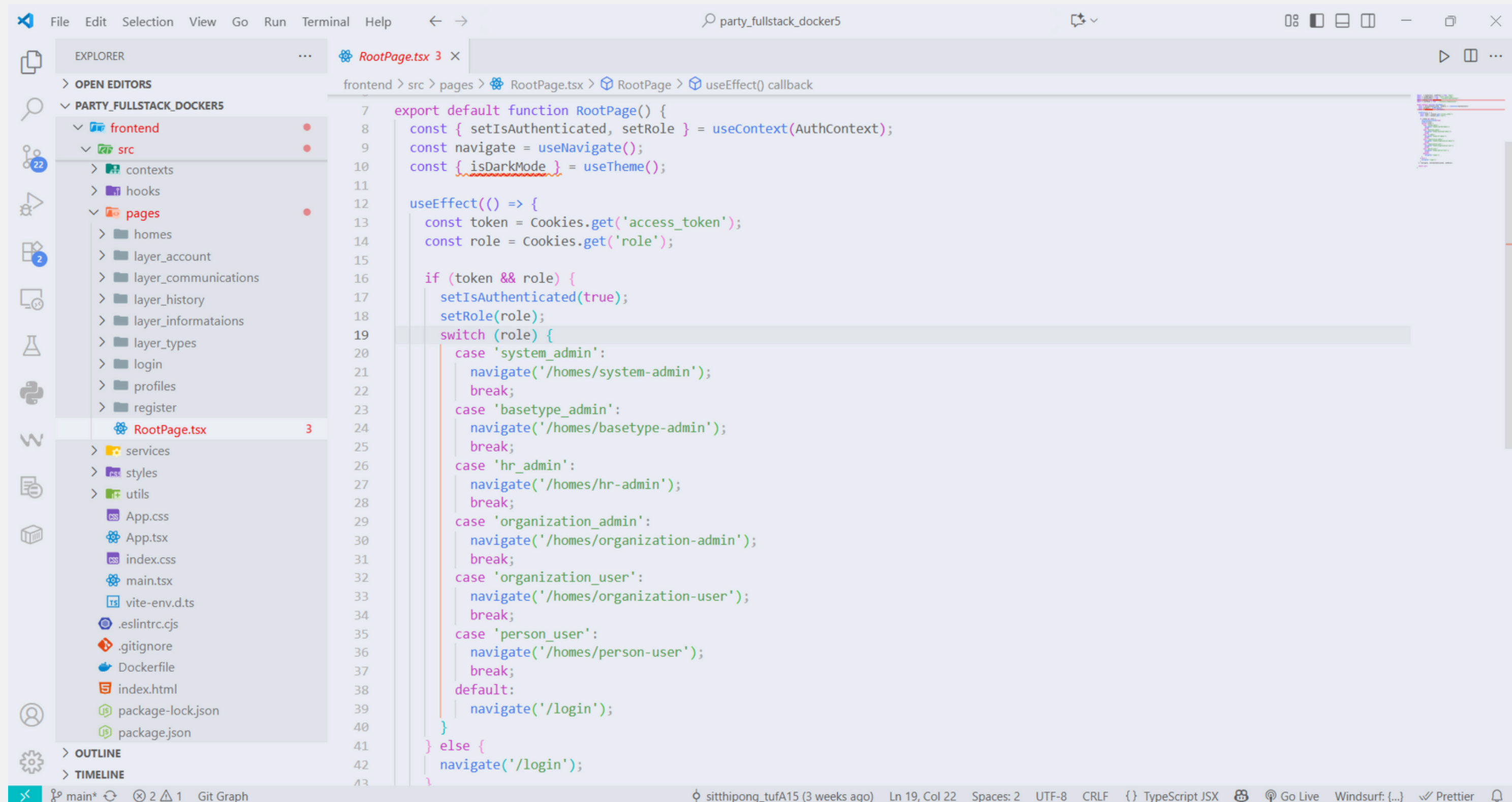
Frontend



- จัดวาง folder ตามแนวทาง Born to Dev จาก YouTube
- มีบาง folder เผื่อไว้ แม้ยังไม่ได้ใช้งาน
- Components เก็บส่วนประกอบเว็บที่ใช้ซ้ำได้ สะดวกต่อการเรียกใช้
- Contexts เก็บ context ทำหน้าที่เหมือน global variable
- Pages เก็บทุกหน้าเว็บของโปรเจกต์
- Services เก็บไฟล์ที่ใช้ติดต่อกับ backend
- Styles เก็บ color scheme และ theme (light/dark)
- Utils เก็บฟังก์ชันทั่วไป เช่น จัดการวันที่และเวลา
- main.tsx ทำหน้าที่ router จัดการ mapping path ไปยังหน้าเว็บ
- RootPage.tsx หน้าแรก πουผู้ใช้เห็น นำทางไป dashboard ตามผู้ใช้ที่ล็อกอิน

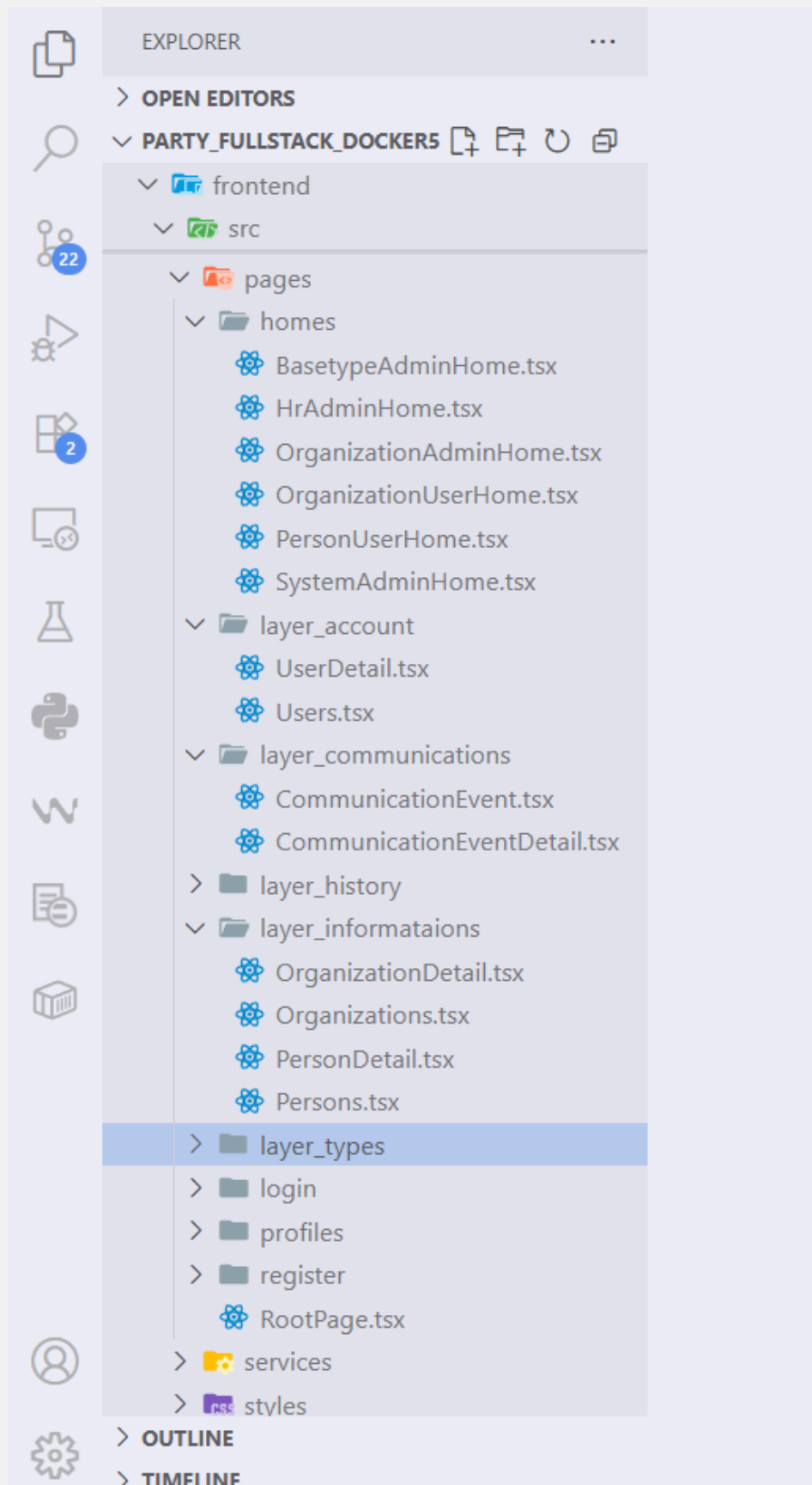

```
72 const unprotectedroutes: UnprotectedRoute[] = [
73   { path: '/register', component: lazy(() => import( './pages/register/register' )) },
74 ];
75
76 createRoot(document.getElementById('root')!).render(
77   <React.StrictMode>
78     <ThemeProvider>
79       <AuthProvider>
80         <BrowserRouter>
81           <Suspense fallback={<Loading />}>
82             <Routes>
83               {unprotectedroutes.map((route) => (
84                 <Route
85                   key={route.path}
86                   path={route.path}
87                   element={<route.component />}
88                 />
89               ))}
90             {protectedroutes.map((route) => (
91               <Route
92                 key={route.path}
93                 path={route.path}
94                 element={
95                   <ProtectedRoute>
96                     <route.component />
97                   </ProtectedRoute>
98                 }
99               />
100             ))}
101             <Route path="*" element={<h1>404 Not Found</h1>} />
102           </Routes>
103         </Suspense>
104       </BrowserRouter>
105     </ThemeProvider>
106   </React.StrictMode>
107 );
```

main.tsx เป็นไฟล์เริ่มต้นสำหรับรัน frontend container



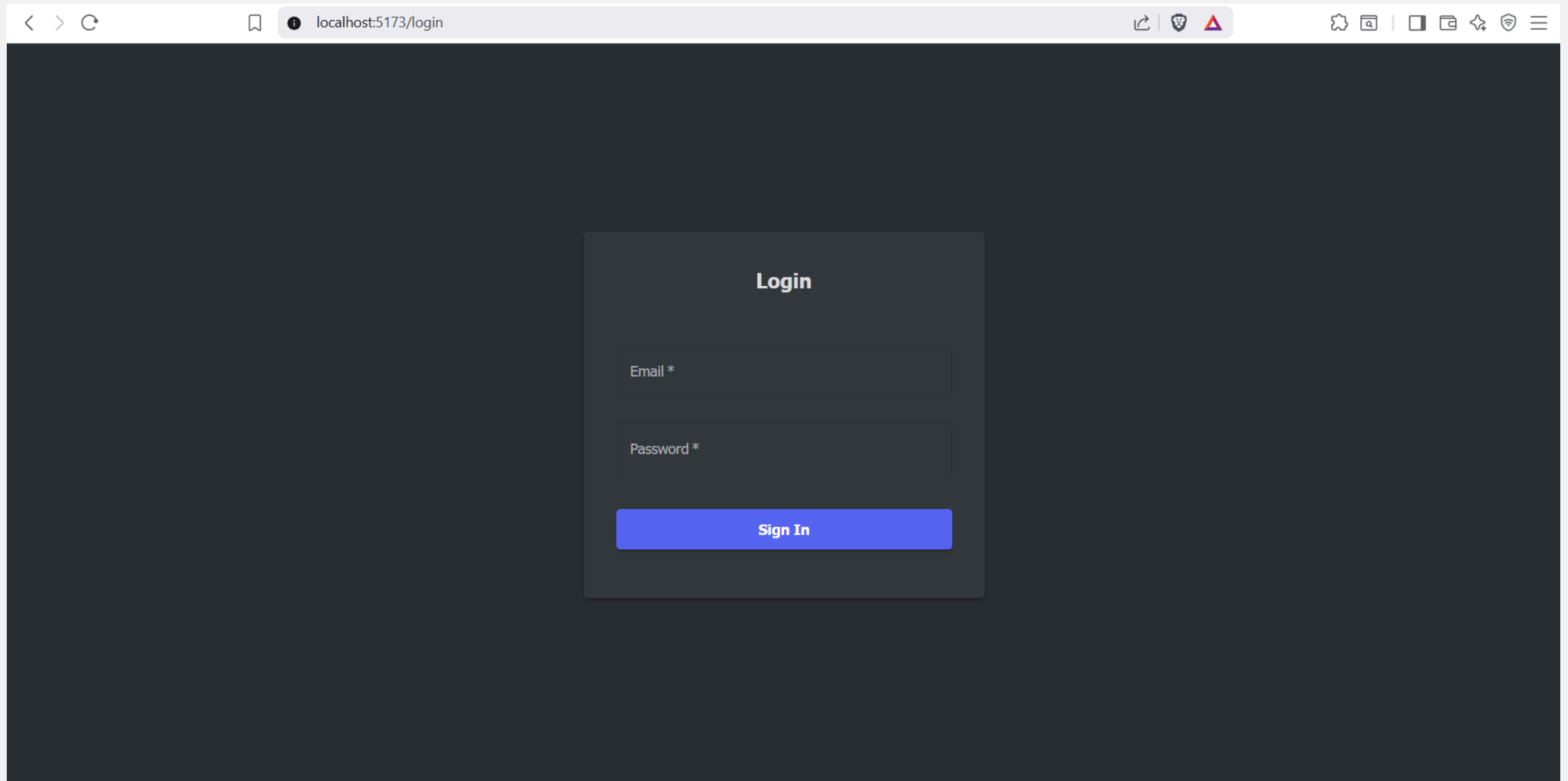
```
7 export default function RootPage() {
8   const { setIsAuthenticated, setRole } = useContext(AuthContext);
9   const navigate = useNavigate();
10  const { isDarkMode } = useTheme();
11
12  useEffect(() => {
13    const token = Cookies.get('access_token');
14    const role = Cookies.get('role');
15
16    if (token && role) {
17      setIsAuthenticated(true);
18      setRole(role);
19      switch (role) {
20        case 'system_admin':
21          navigate('/homes/system-admin');
22          break;
23        case 'basetype_admin':
24          navigate('/homes/basetype-admin');
25          break;
26        case 'hr_admin':
27          navigate('/homes/hr-admin');
28          break;
29        case 'organization_admin':
30          navigate('/homes/organization-admin');
31          break;
32        case 'organization_user':
33          navigate('/homes/organization-user');
34          break;
35        case 'person_user':
36          navigate('/homes/person-user');
37          break;
38        default:
39          navigate('/login');
40      }
41    } else {
42      navigate('/login');
43    }
44  });
45 }
```

RootPage เป็นหน้าแรกที่ใช้พบ ทำหน้าที่ redirect ไปยังหน้าอื่นๆ ตามลำดับ

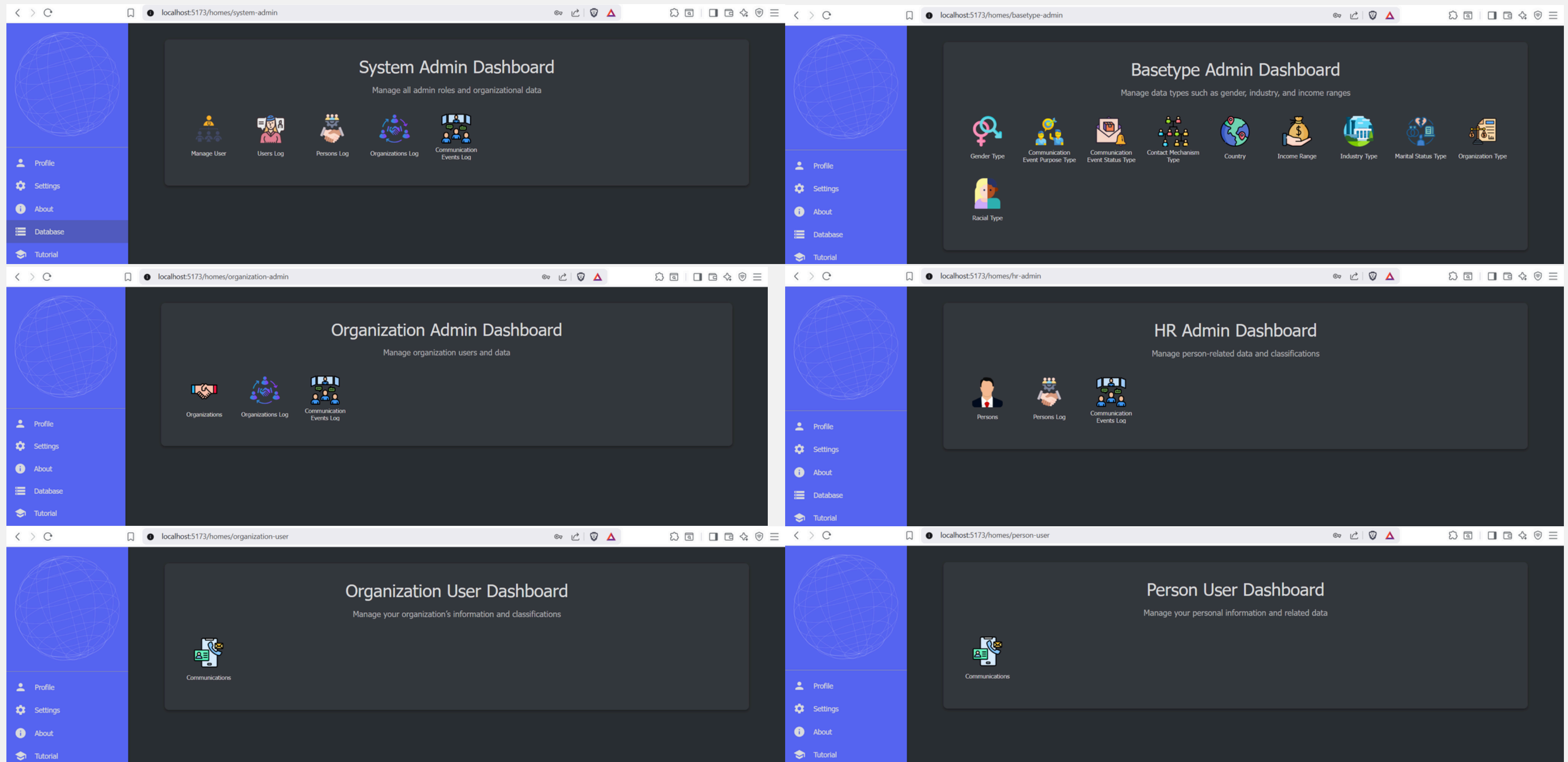


- Folder Page เก็บ layer ต่างๆ ของหน้าเว็บ
- Homes รวมหน้า Home เฉพาะสำหรับแต่ละ role
- Layer Account จัดการ CRUD สำหรับ account ในแอป
- Layer Communication จัดการ CRUD การสื่อสาร
- Layer History เก็บหน้าแสดงประวัติทั้งหมด
- Layer Information จัดการ CRUD สำหรับ organization และ person
- Layer Types จัดการ CRUD ข้อมูล base type
- Profile เก็บหน้า profile ของแต่ละ role

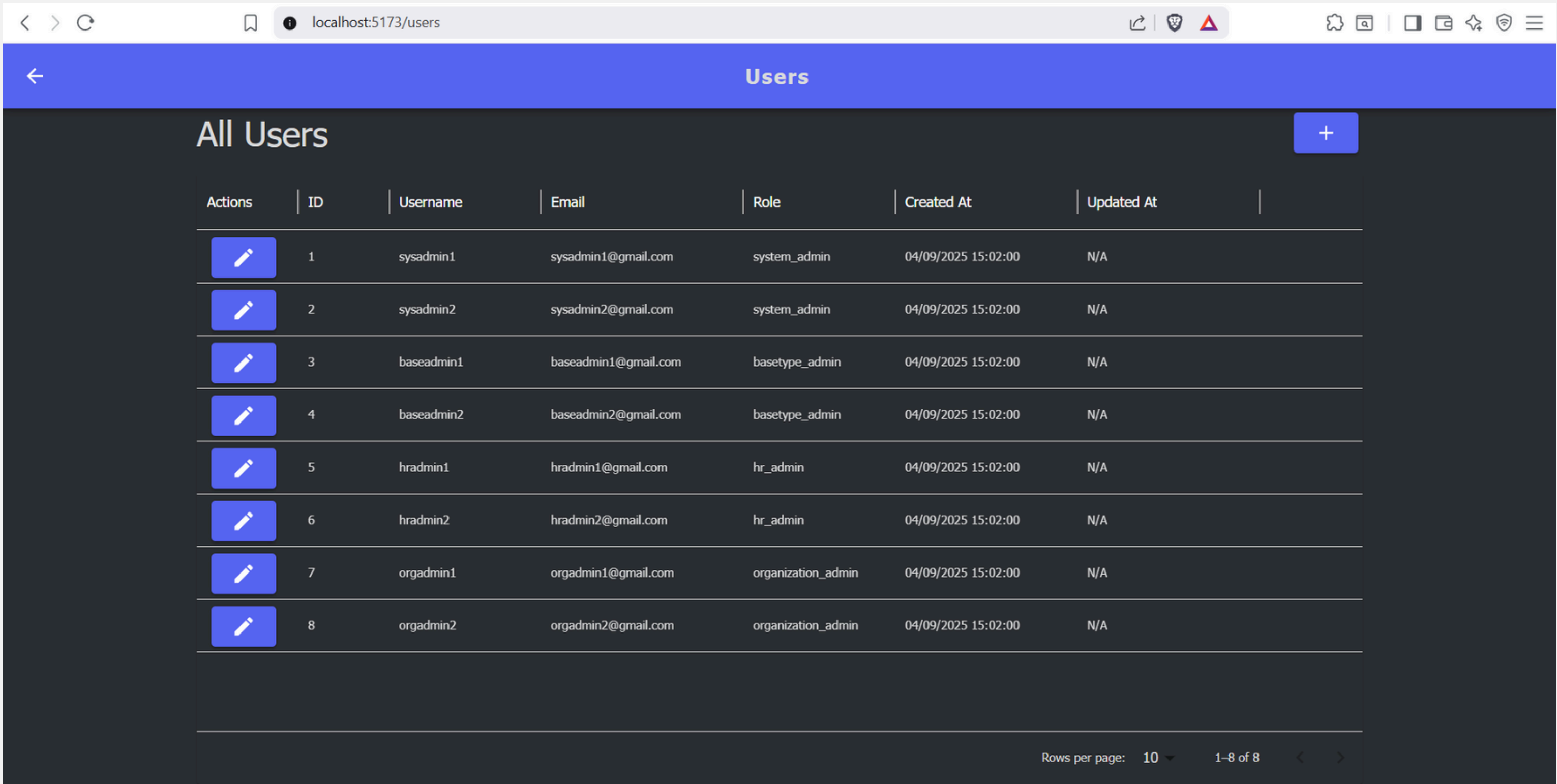
ตัวอย่างหน้าเว็บ



หน้า Login



Dashboard สำหรับผู้ใช้ทั้ง 6 บทบาท



System Admin จัดการ Admins

localhost:5173/persons

Persons

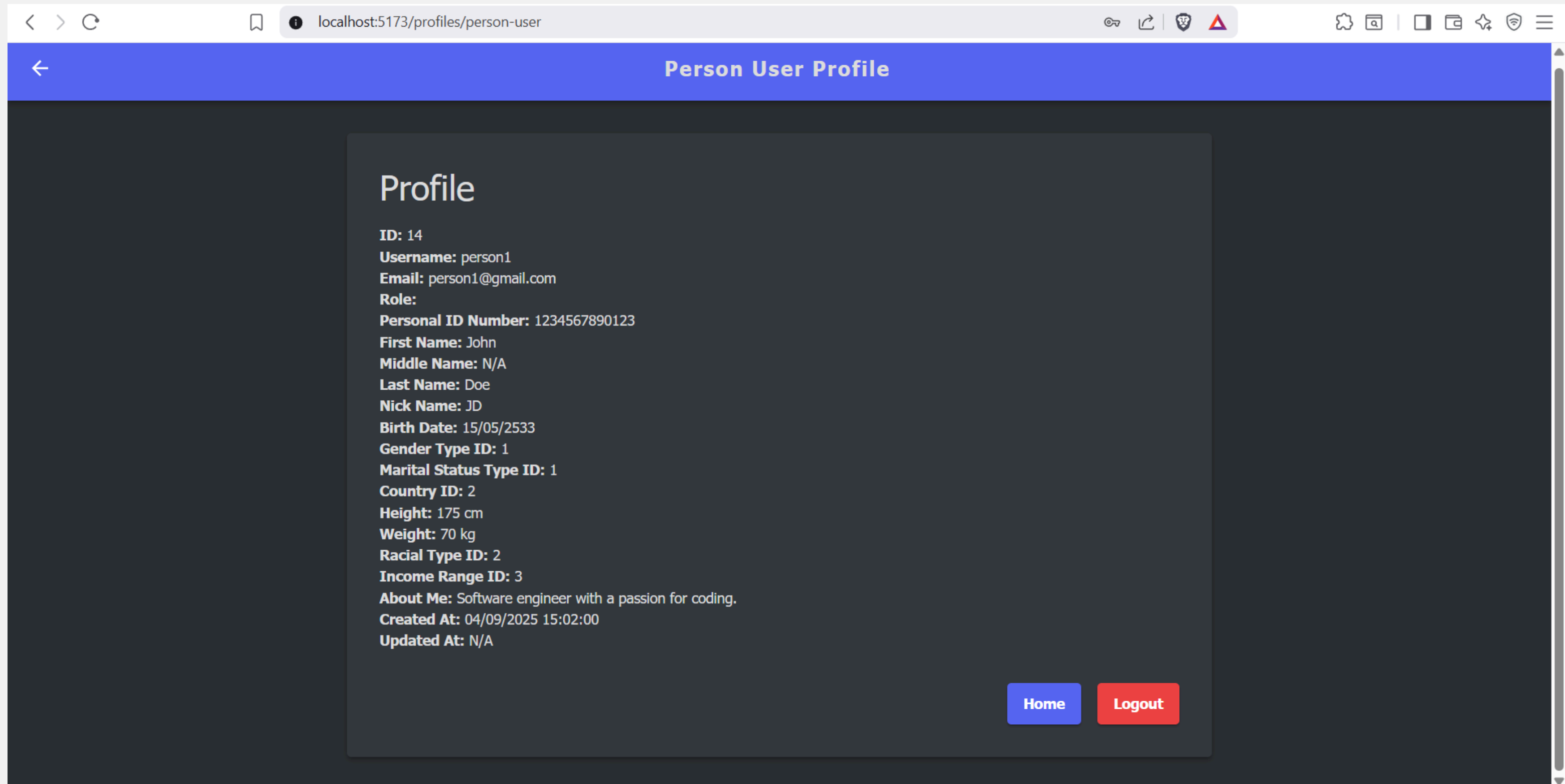
All Persons

Actions	ID	Username	Email	Personal ID	First Name	Middle Name	Last Name	
	14	person1	person1@gmail.com	1234567890123	John	N/A	Doe	
	15	person2	person2@gmail.com	2345678901234	Jane	Ann	Smith	
	16	person3	person3@gmail.com	3456789012345	Mike	N/A	Brown	
	17	person4	person4@gmail.com	4567890123456	Anna	N/A	Lee	
	18	person5	person5@gmail.com	5678901234567	Somchai	N/A	Jaidee	

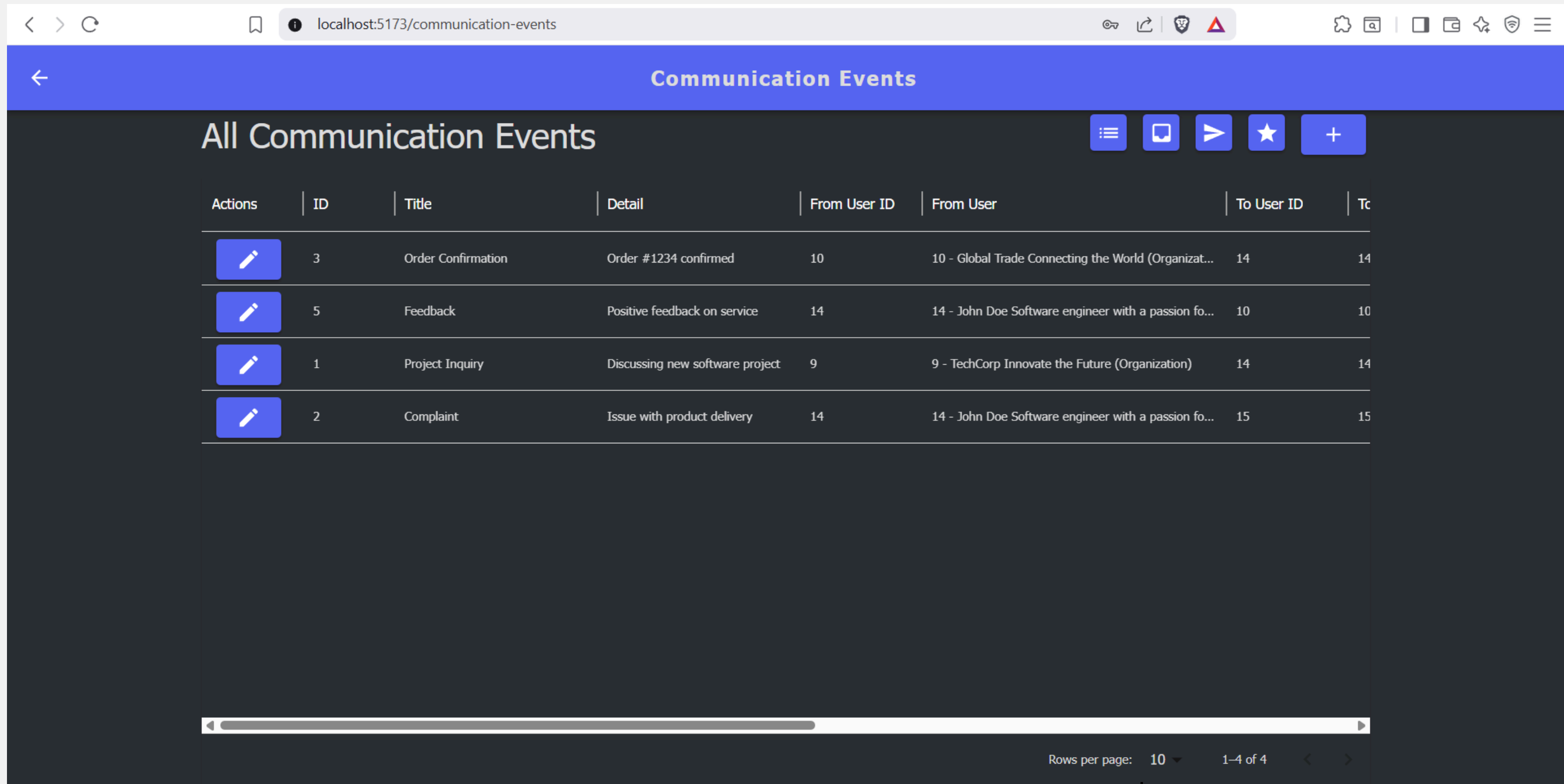
Rows per page: 10

1-5 of 5

HR Admin จัดการ Persons



Person สามารถดู Profile ตัวเองได้



Person และ Organization จัดการเกี่ยวกับการสื่อสารได้

ฟีเจอร์สำหรับ ผู้ใช้งานที่เป็น Person และ Organization

localhost:5173/communication-events/create

Create Communication Event

Create New Communication Event

Title

Detail

To User

Contact Mechanism Type

Status

Create Communication

localhost:5173/communication-events

Communication Events

All Communication Events

Actions	ID	Title	Detail	From User ID	From User	To User ID	To User
	3	Order Confirmation	Order #1234 confirmed	10	10 - Global Trade Connecting the World (Organization)	14	14 - John Doe Software engineer with a passion for...
	5	Feedback	Positive feedback on service	14	14 - John Doe Software engineer with a passion for...	10	10 - Global Trade Connecting the World (Organization)
	1	Project Inquiry	Discussing new software project	9	9 - TechCorp Innovate the Future (Organization)	14	14 - John Doe Software engineer with a passion for...
	2	Complaint	Issue with product delivery	14	14 - John Doe Software engineer with a passion for...	15	15 - Jane Smith Product Manager at Global Trade

Rows per page: 10

1-4 of 4

Read Communication

localhost:5173/communication-events/3

Edit Communication Event

ID

3

Title

Order Confirmation

Detail

Order #1234 confirmed

To User

14 - John Doe Software engineer with a passion for coding. (Person)

Contact Mechanism Type

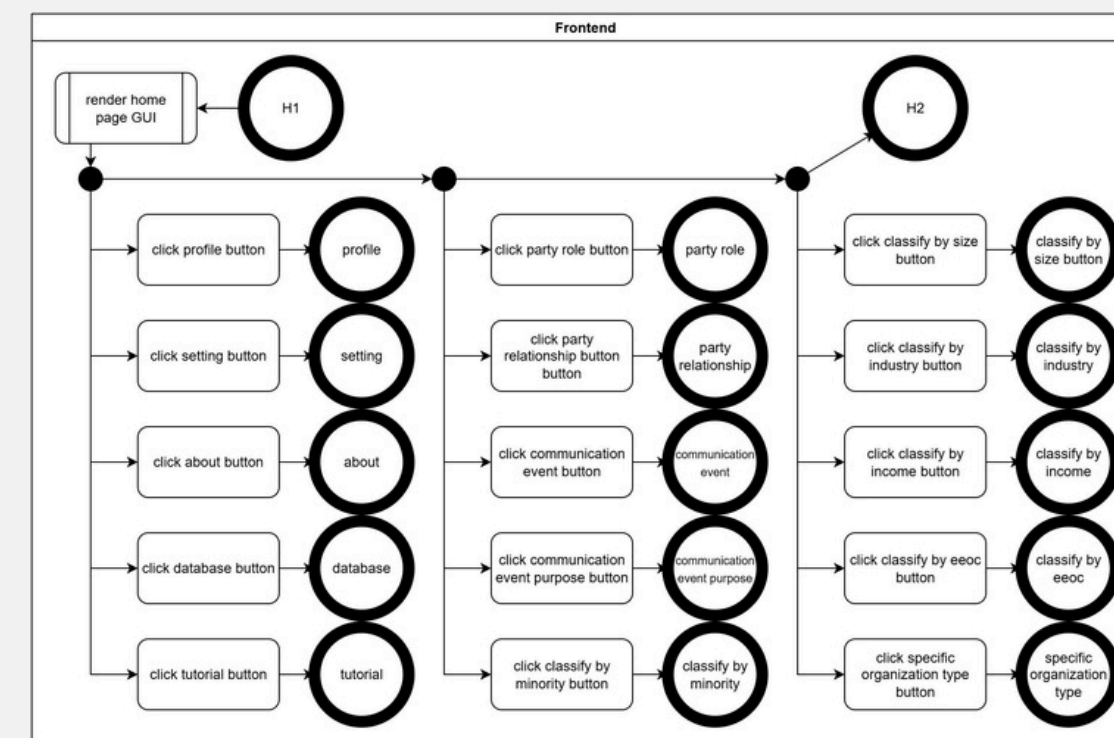
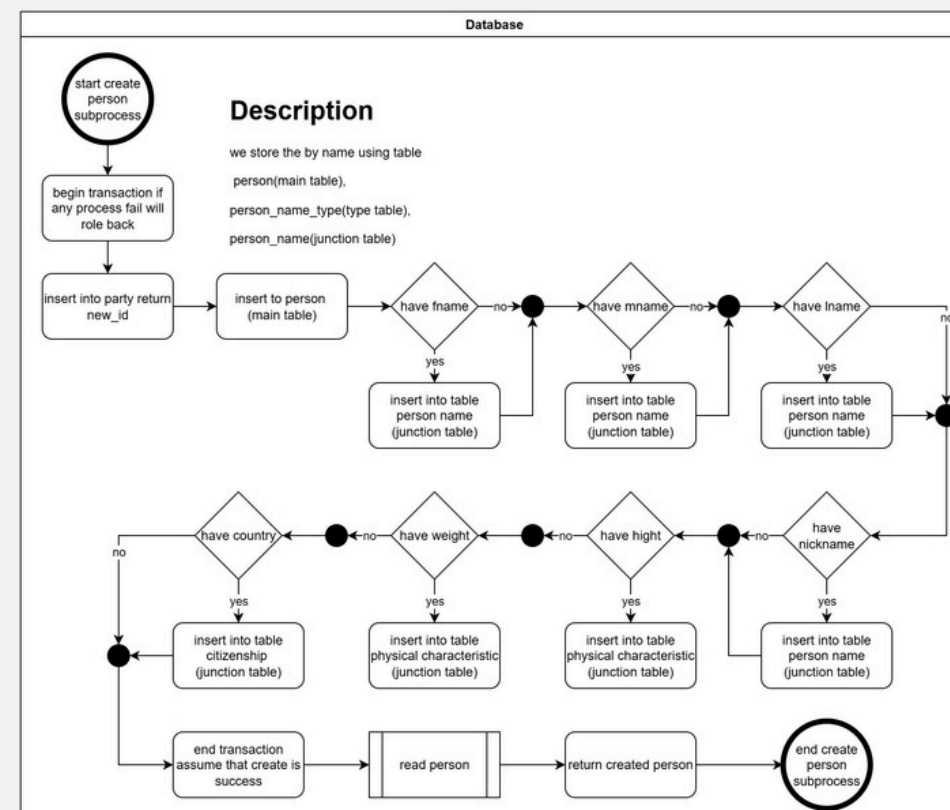
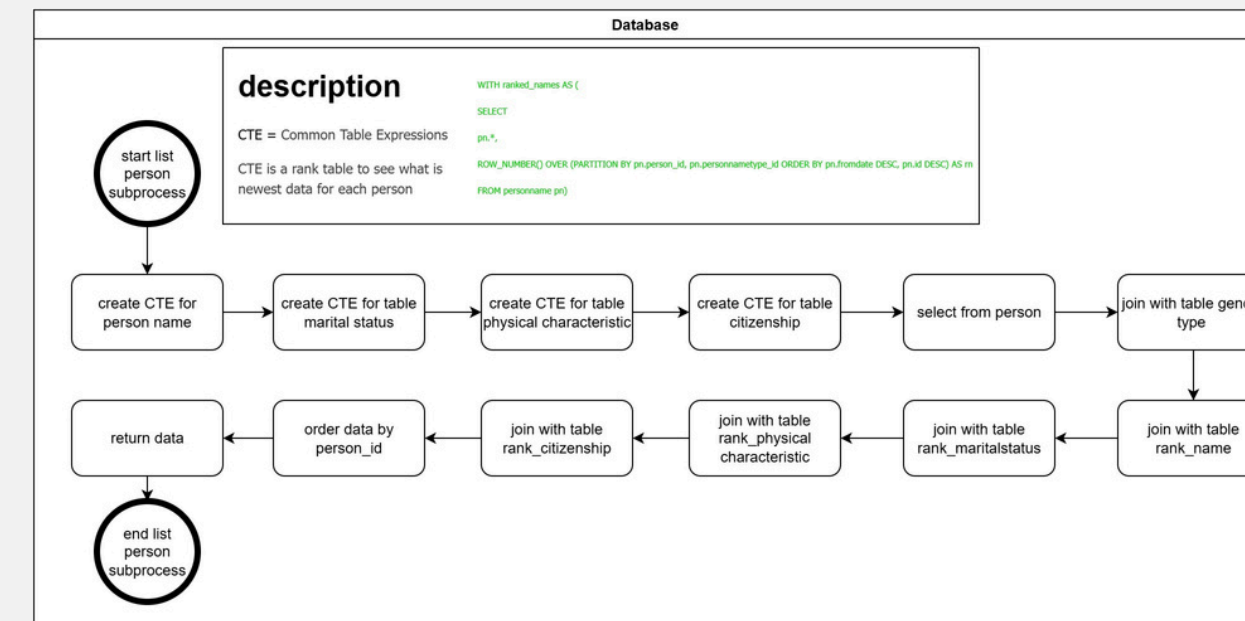
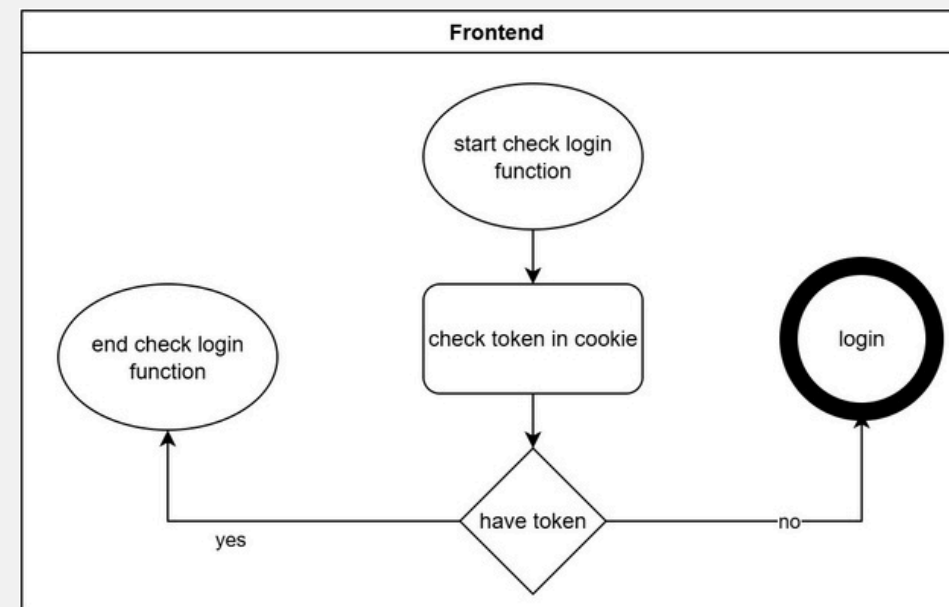
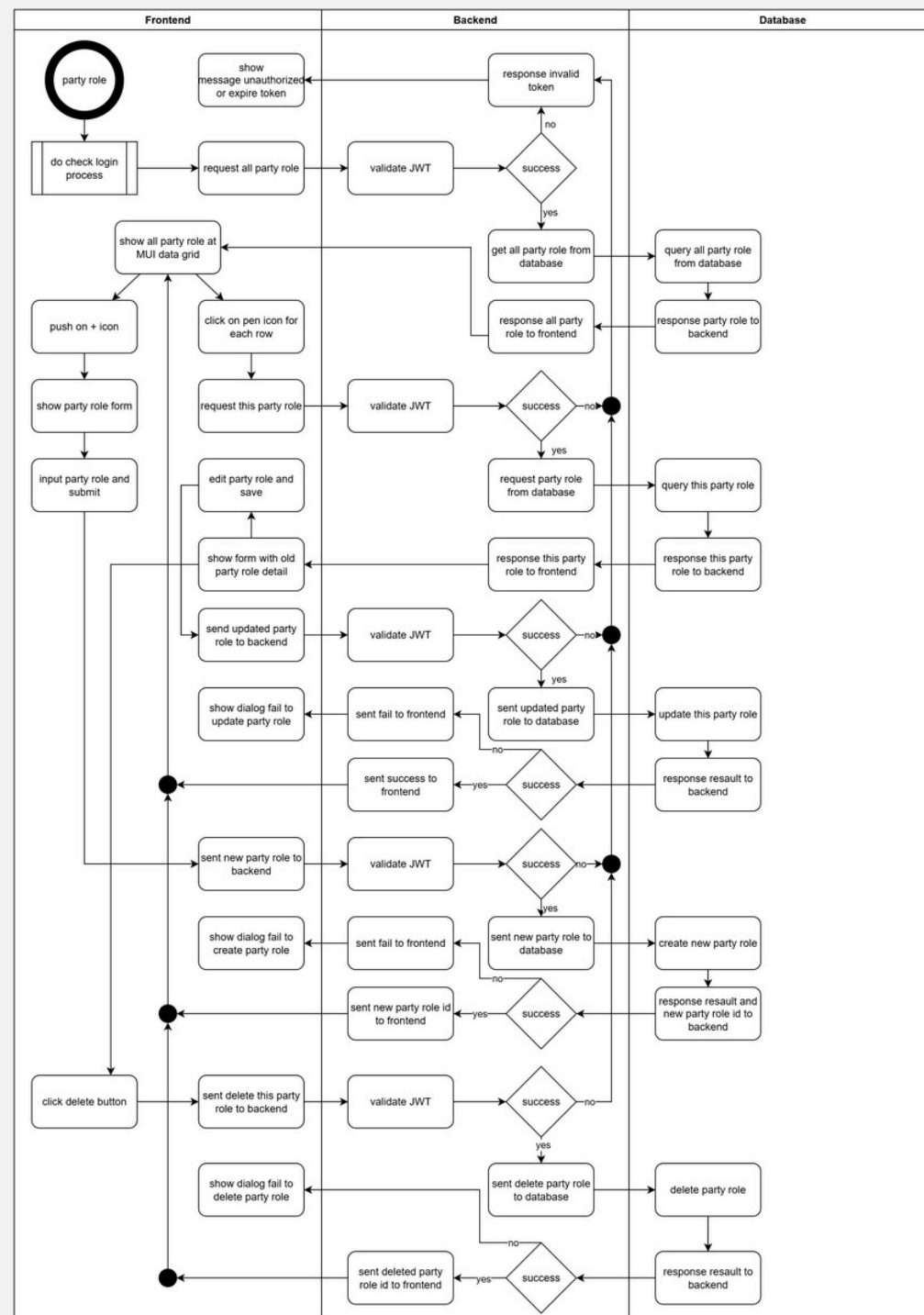
Address

Status

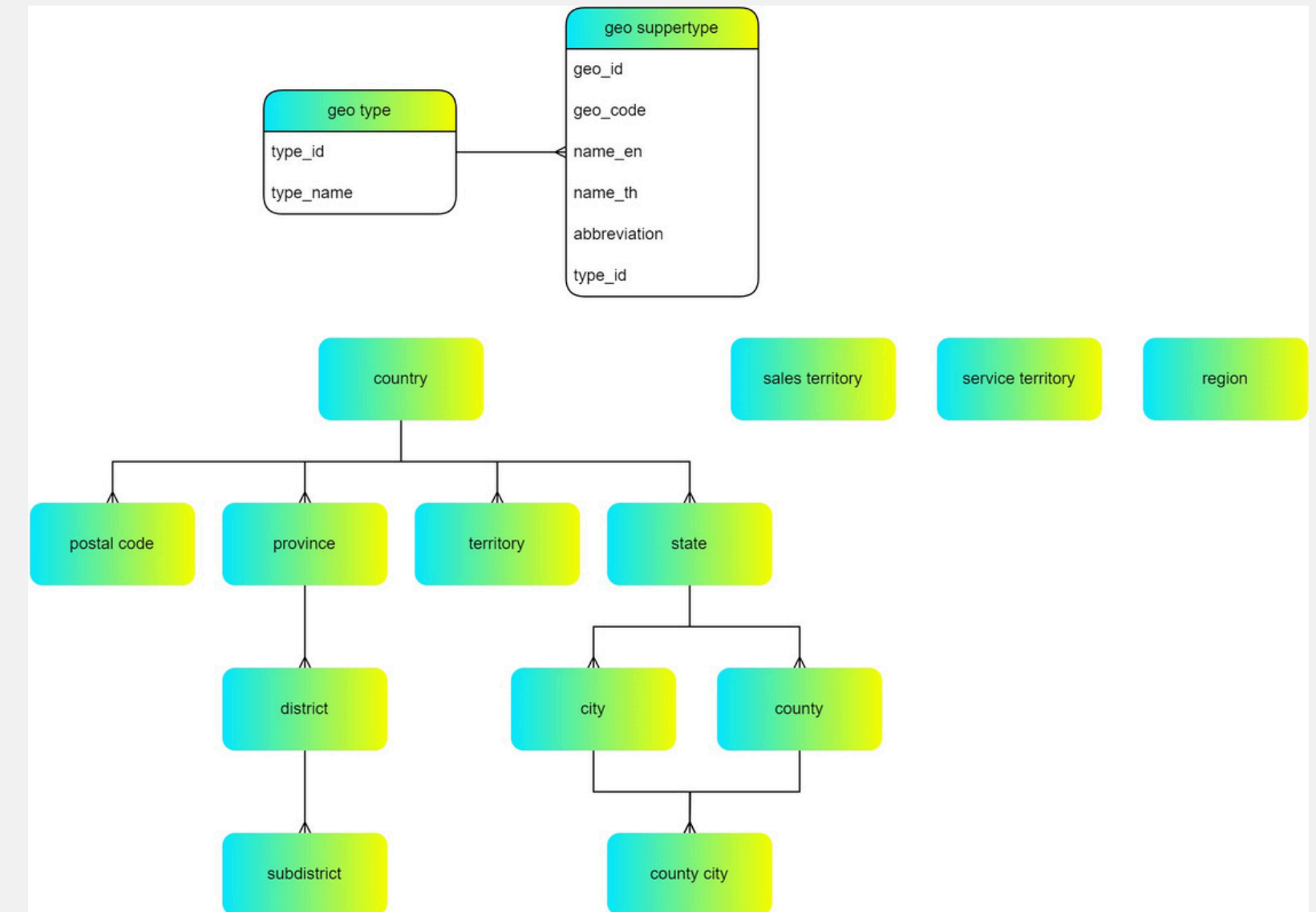
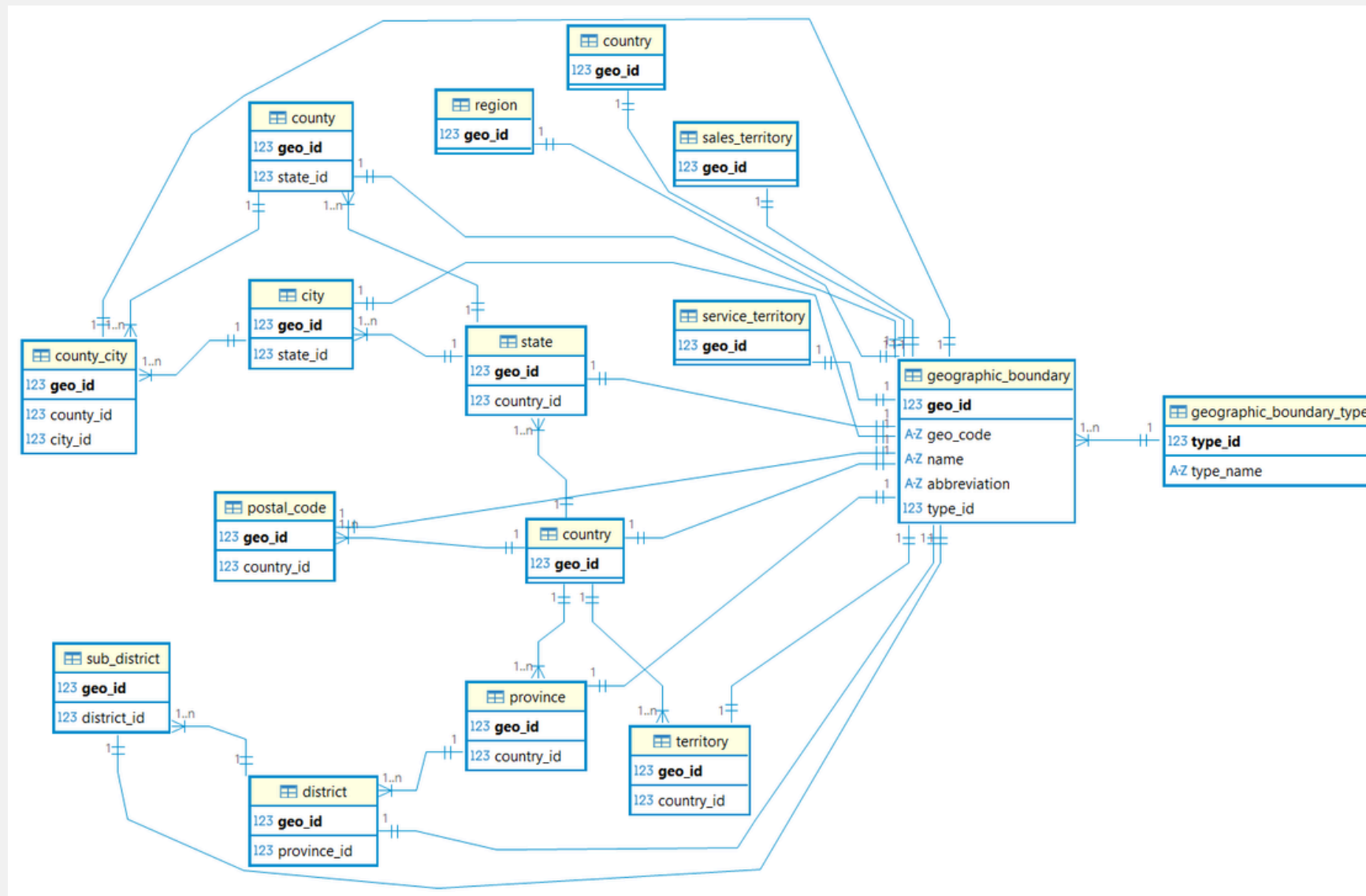
Completed

Update and Delete Communication

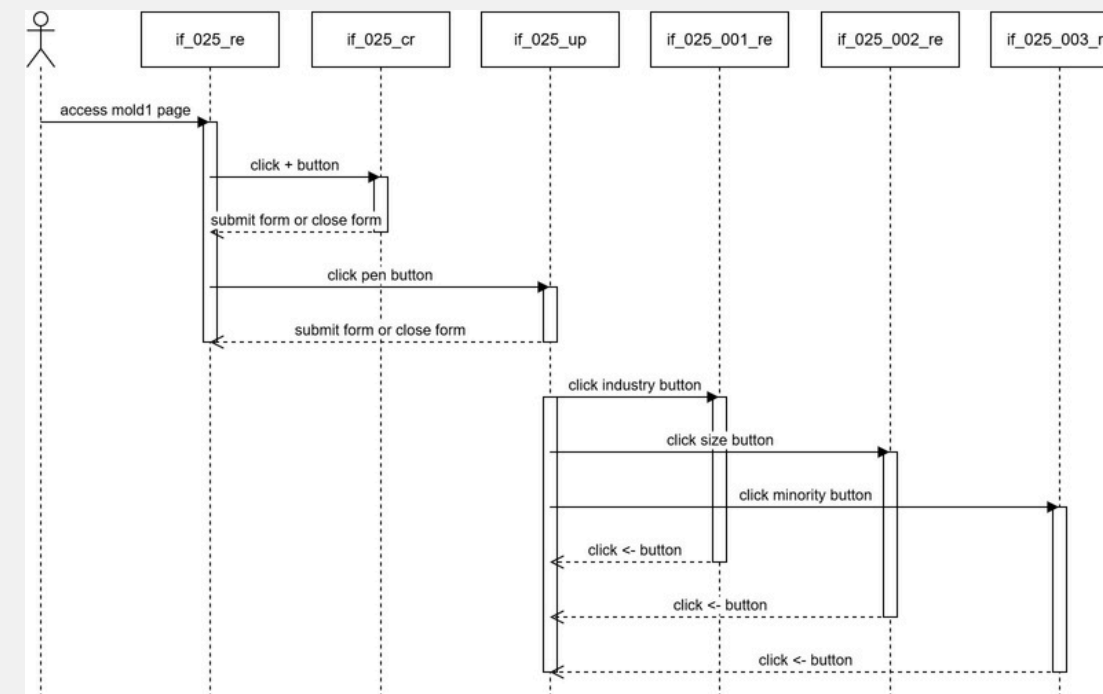
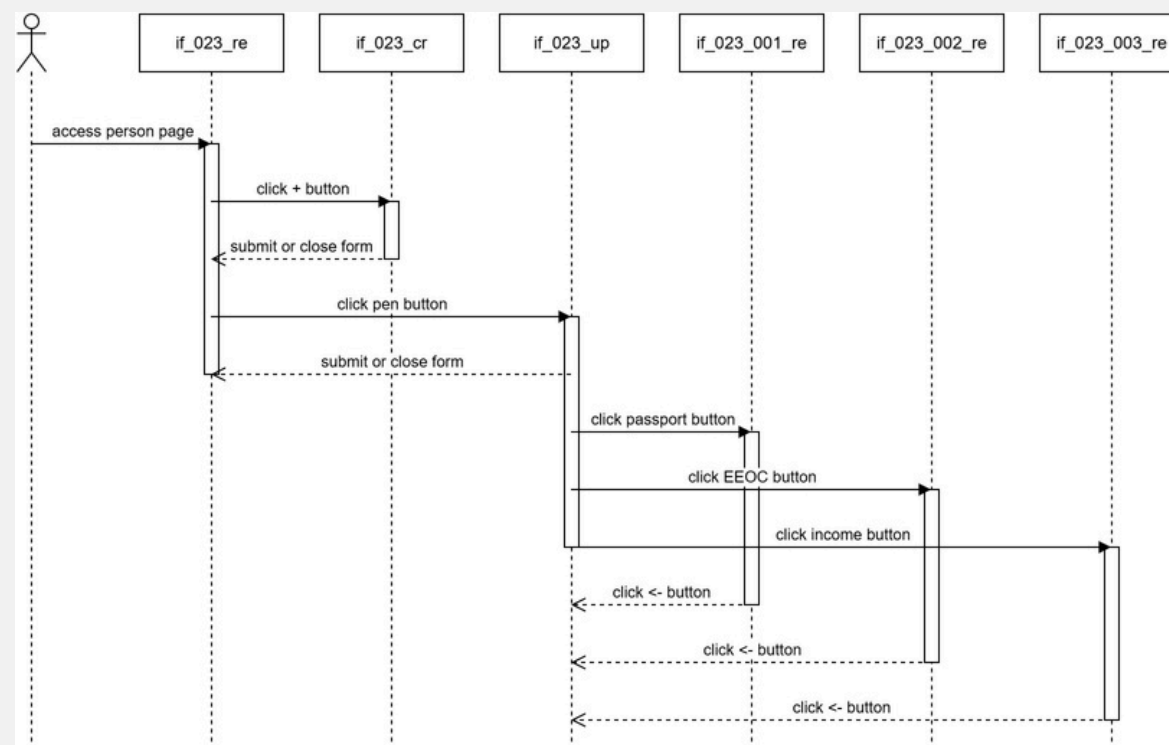
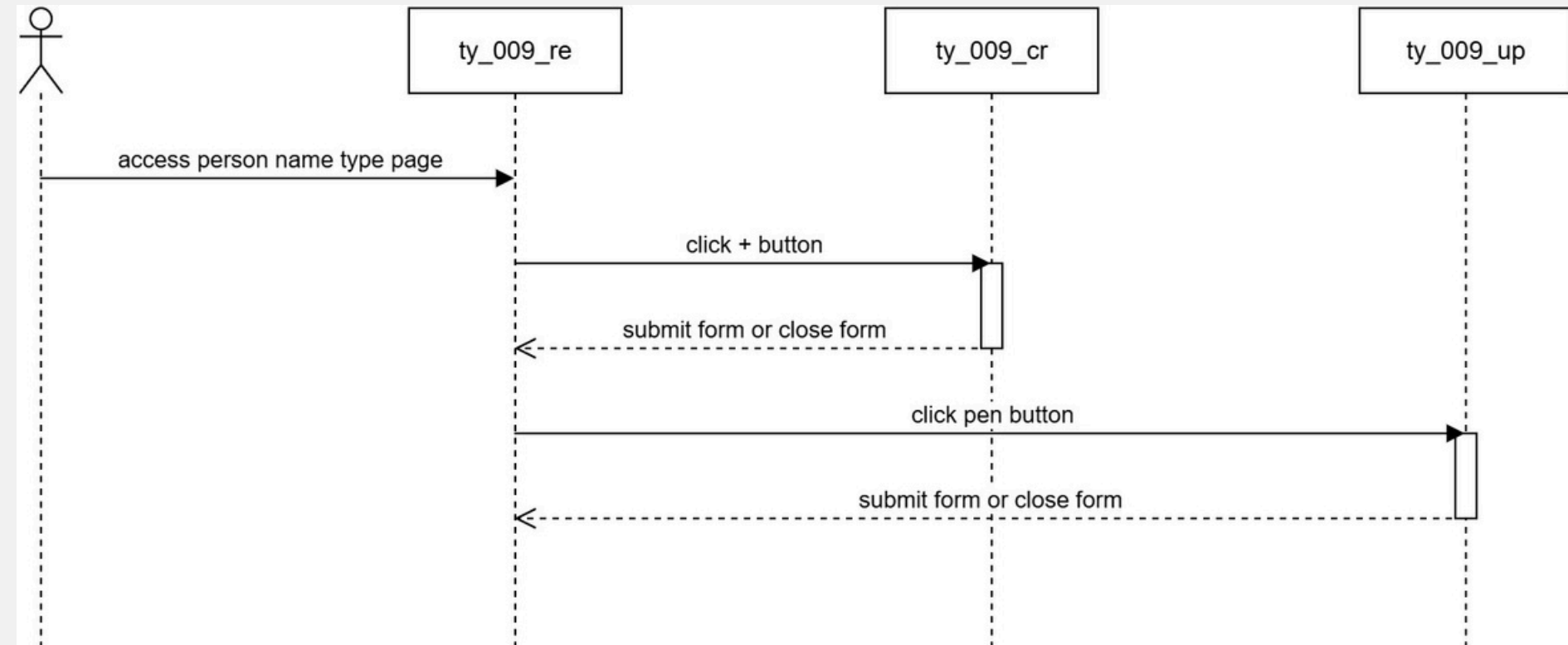
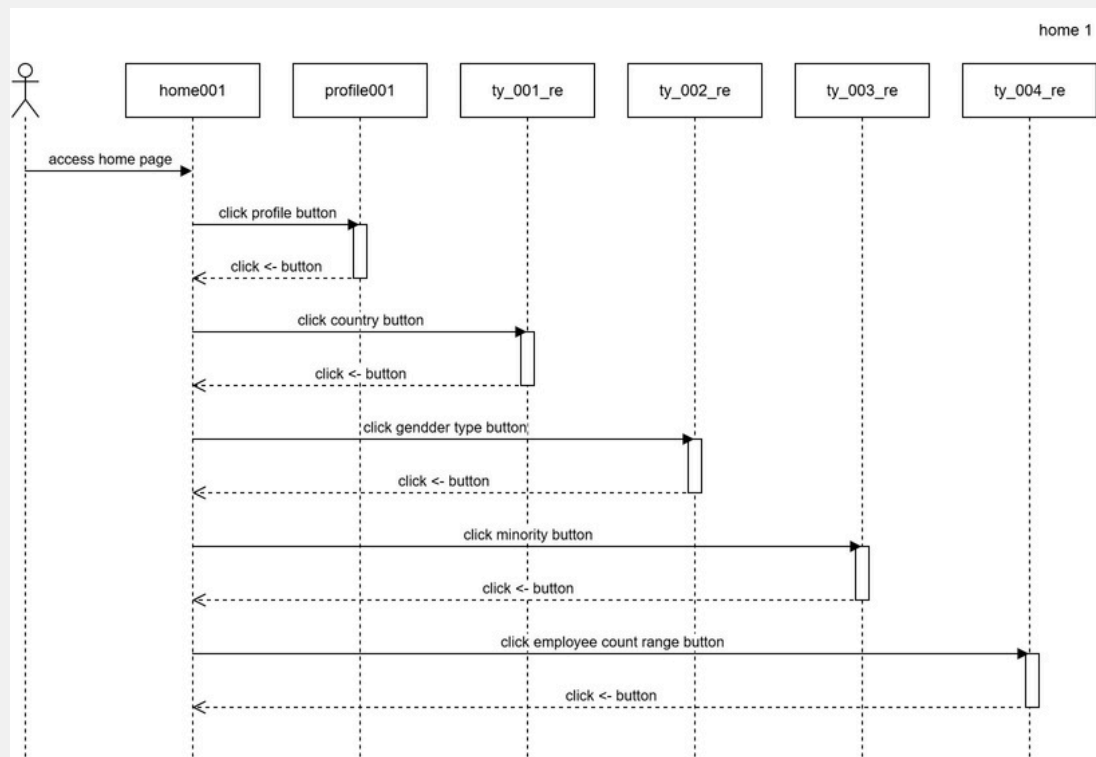
Software Document



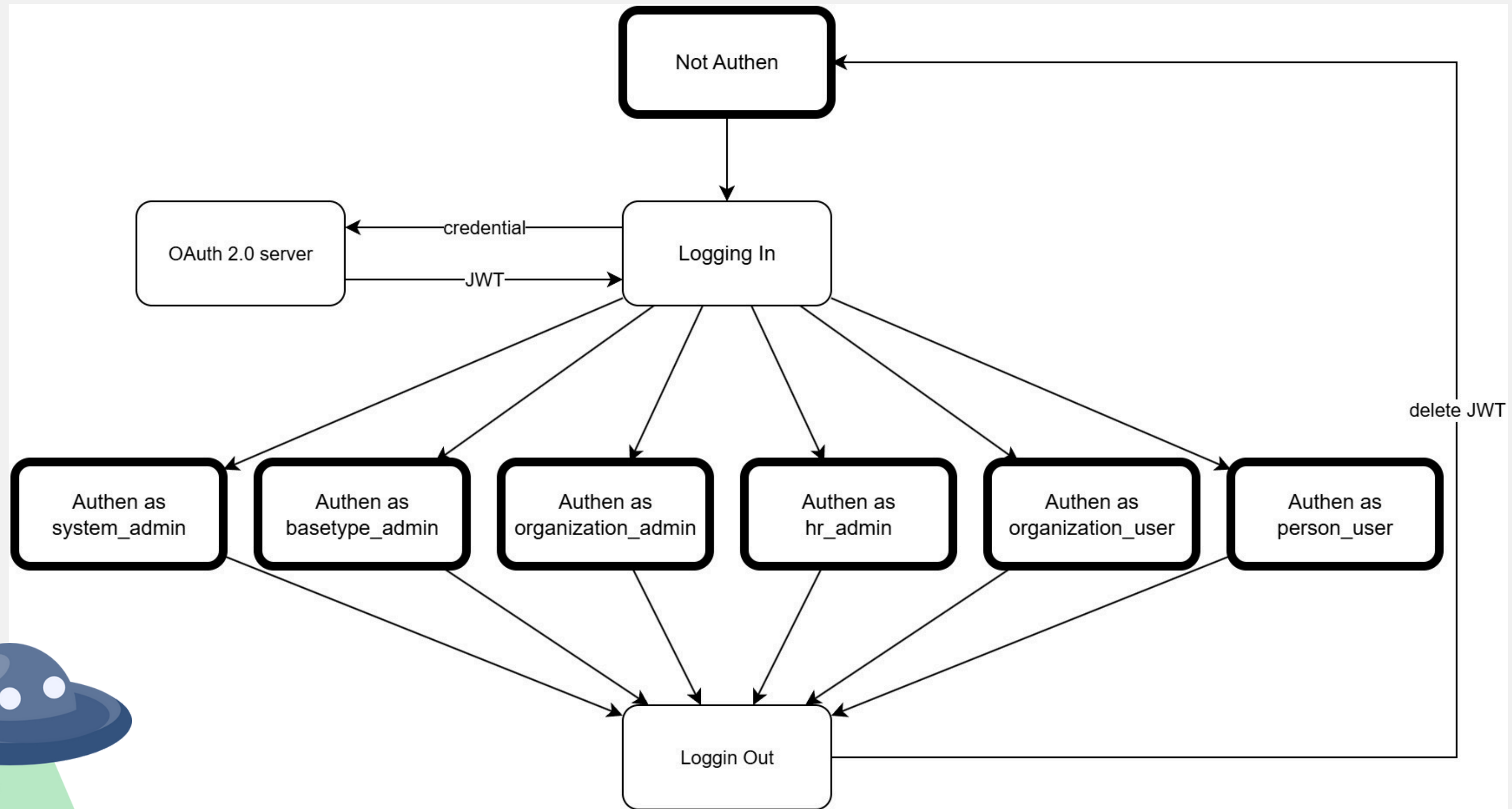
ตัวอย่าง Activity Diagram



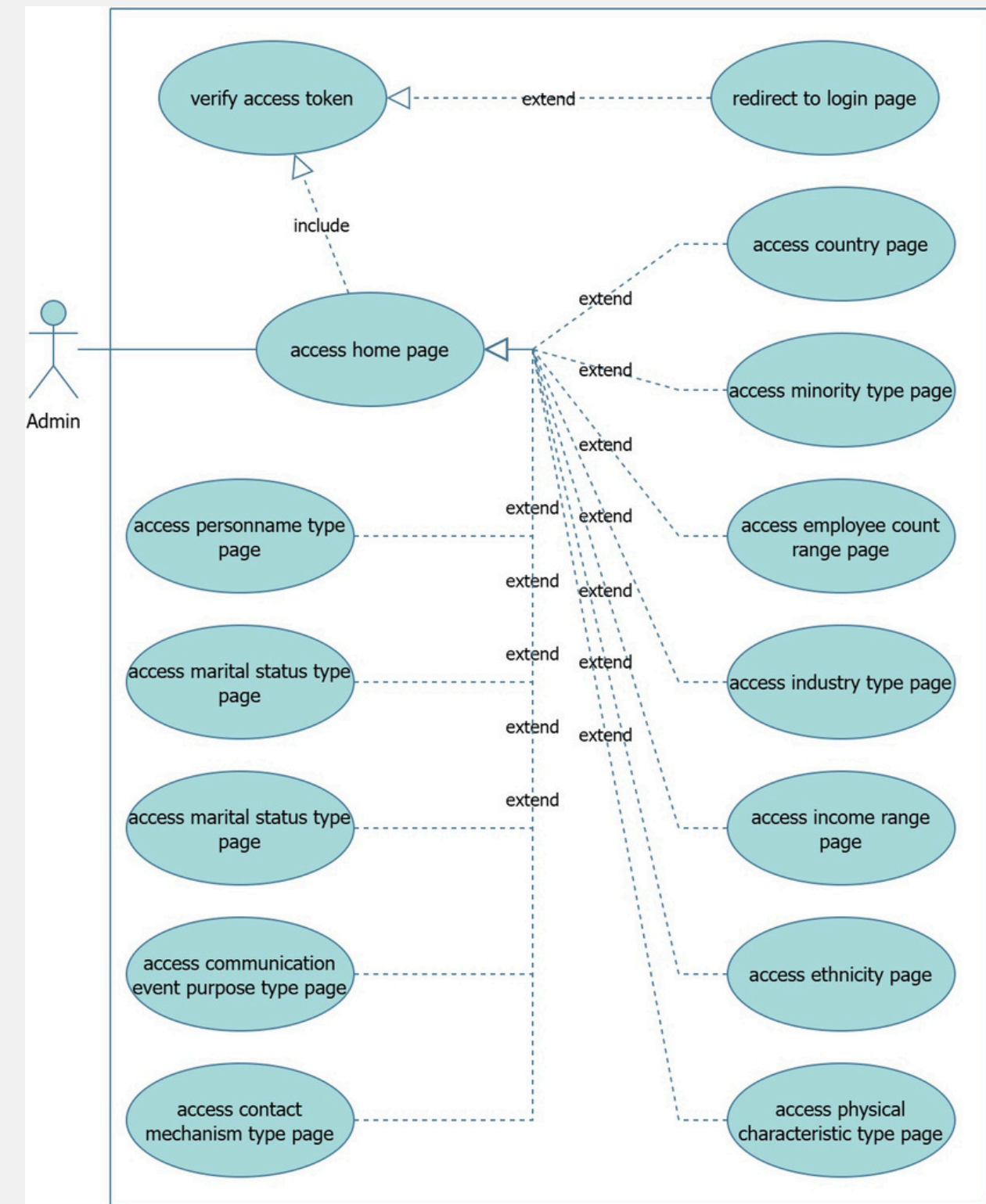
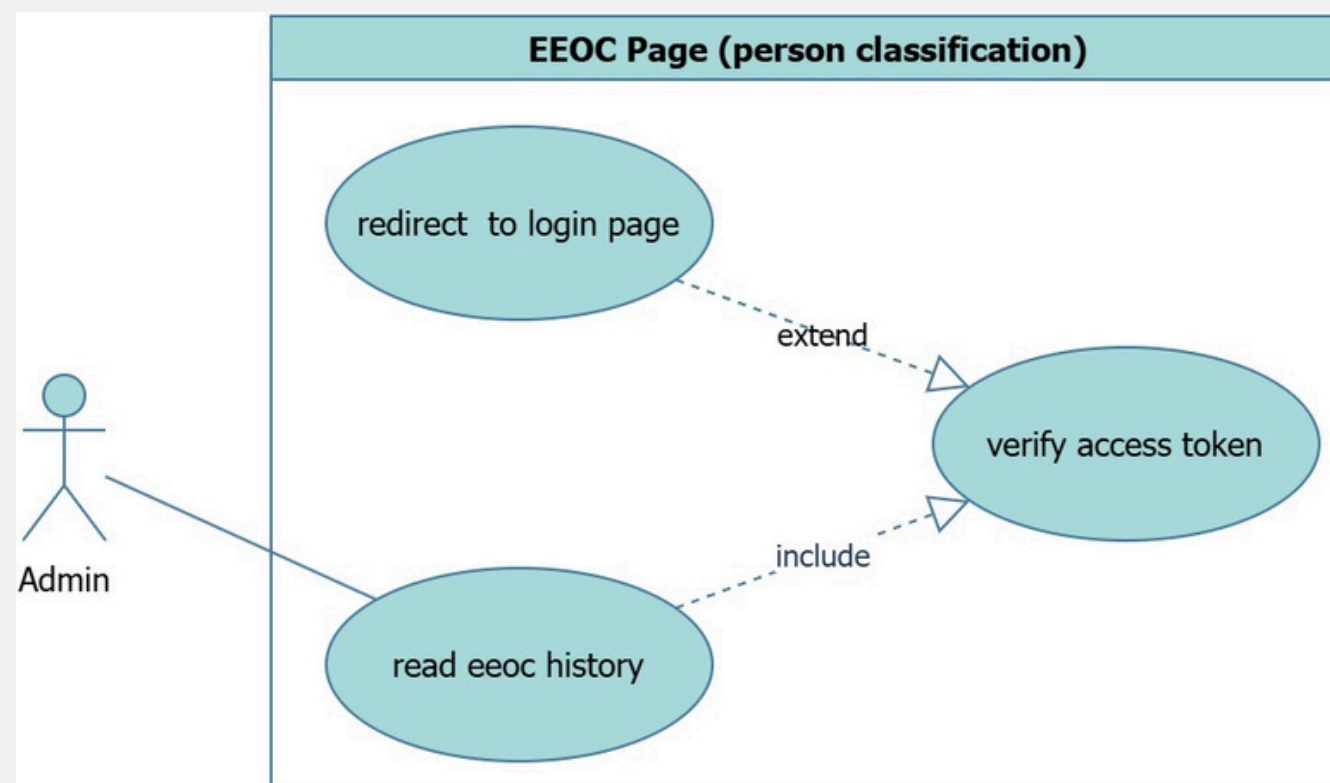
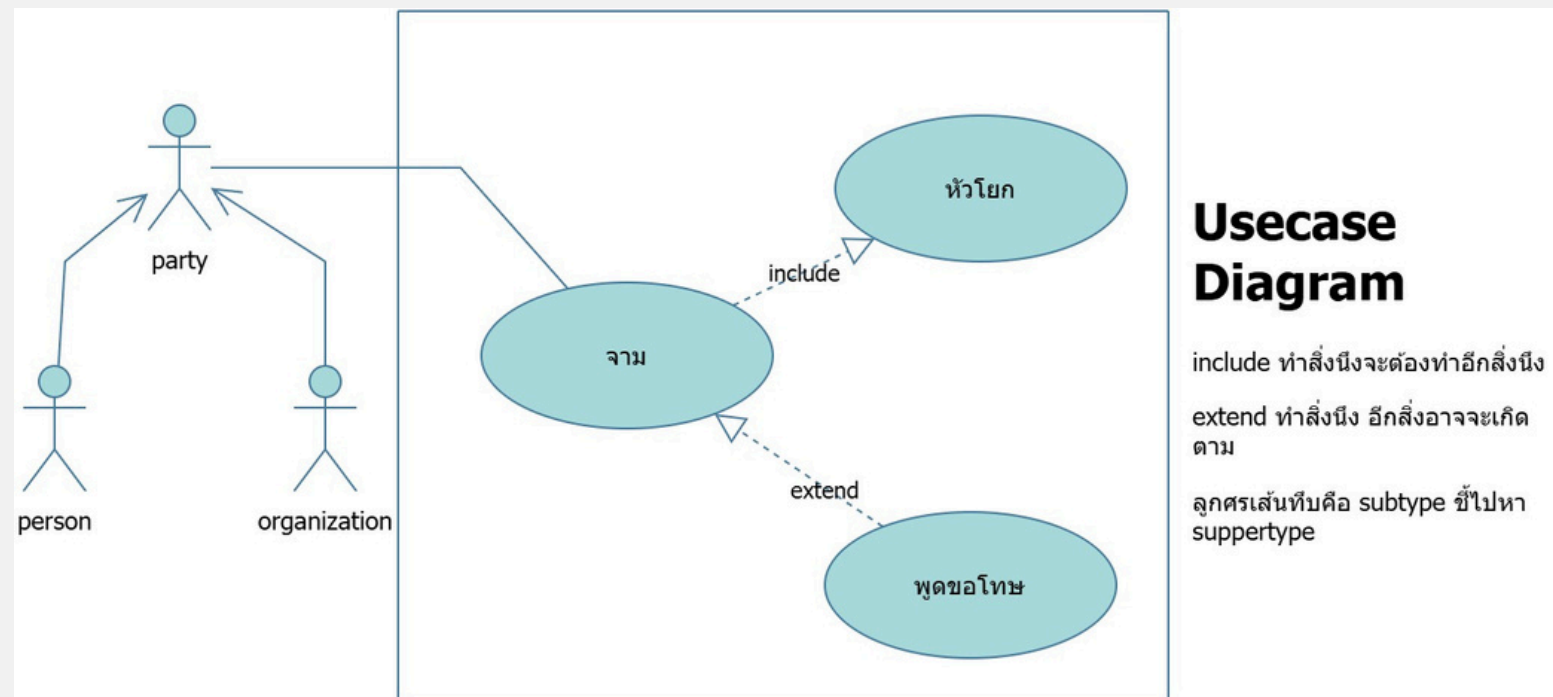
ตัวอย่าง ERD (Geo Graphic Boundary)



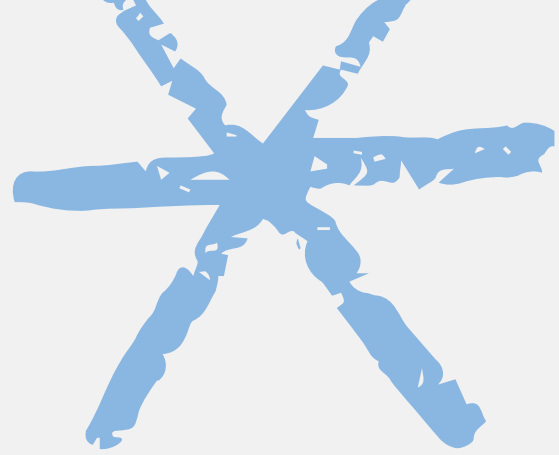
ตัวอย่าง Sequence Diagram (Party Communication)



ตัวอย่าง State Machine Diagram (ล่าสุด)



ตัวอย่าง Usecase Diagram



Dev Ops

Summary Layer	Total Lines
Docker	97
Database	349
Database Data CSV	1347
Backend	2333
Frontend	6318
Total	10444

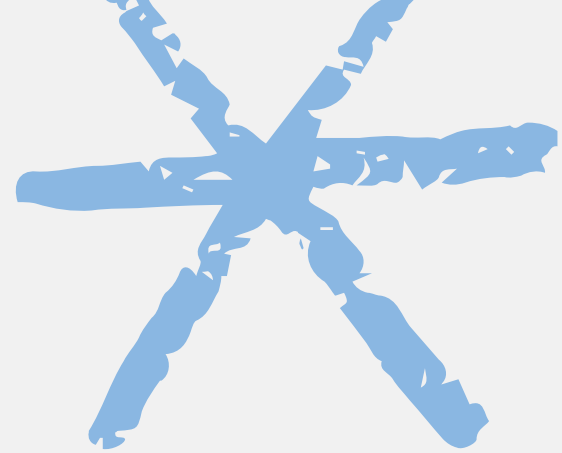
Components

Summary Sub-Layer	Total Lines
Docker - Compose	46
Docker - Backend	31
Docker - Frontend	20
Database - Create	228
Database - Insert Mock	121
Database Data CSV	1347
Backend - Main Files	114
Backend - Schema	355
Backend - Model	1694
Backend - Controller	1170
Frontend - Components	749
Frontend - Contexts	99
Frontend - Pages	4690
Frontend - Services	1049
Frontend - Styles	266
Frontend - Utils	37
Frontend - Main	110
Total	10444



จำนวนโค้ด Project ล่าสุด





Dev Ops

Summary Layer	Total Lines
Docker	97
Database	345
Insert Mock Data	2165
Backend	5127
Frontend	9924
Total	17658

Components

Summary Sub-Layer	Total Lines
Docker - Compose	46
Docker - Backend	31
Docker - Frontend	20
Database - Create Table	328
Database - User Init	17
Insert Mock Data	2165
Backend - Main Files	138
Backend - Schemas	793
Backend - Models	3641
Backend - Controllers	1591
Backend - Config	30
Frontend - Components	2455
Frontend - Contexts	105
Frontend - Pages	6269
Frontend - Services	3600
Frontend - Styles	186
Frontend - Main	352
Total	17658



จำนวนโค้ด Party Communication



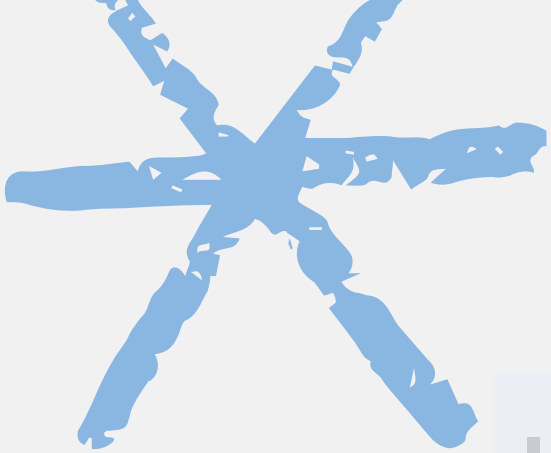
Dev Ops

Summary Layer	Total Lines
Docker	124
Nginx	56
Database	197
Insert Mock Data	20106
Backend	1623
Frontend	4978
Total	27084

Components

Summary Sub-Layer	Total Lines
Docker - Compose	85
Docker - Backend	16
Docker - Frontend	23
Nginx - Config	56
Database - Create Database	197
Insert Mock Data	20106
Backend - Controllers	1496
Backend - Models	313
Backend - Main File	69
Frontend - Components	1410
Frontend - Pages	3326
Frontend - Services	1169
Frontend - Styles	78
Frontend - Main	61
Total	27084

จำนวนโค้ด Geographic Boundary

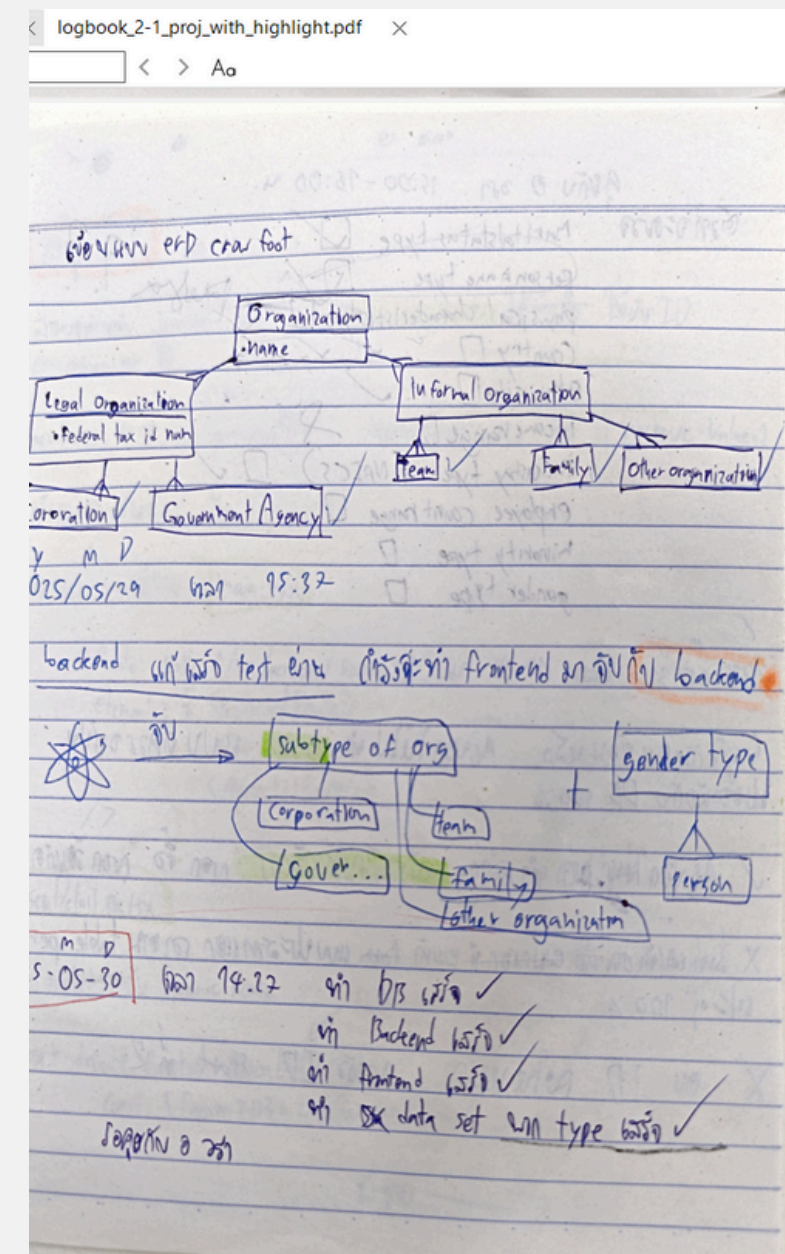
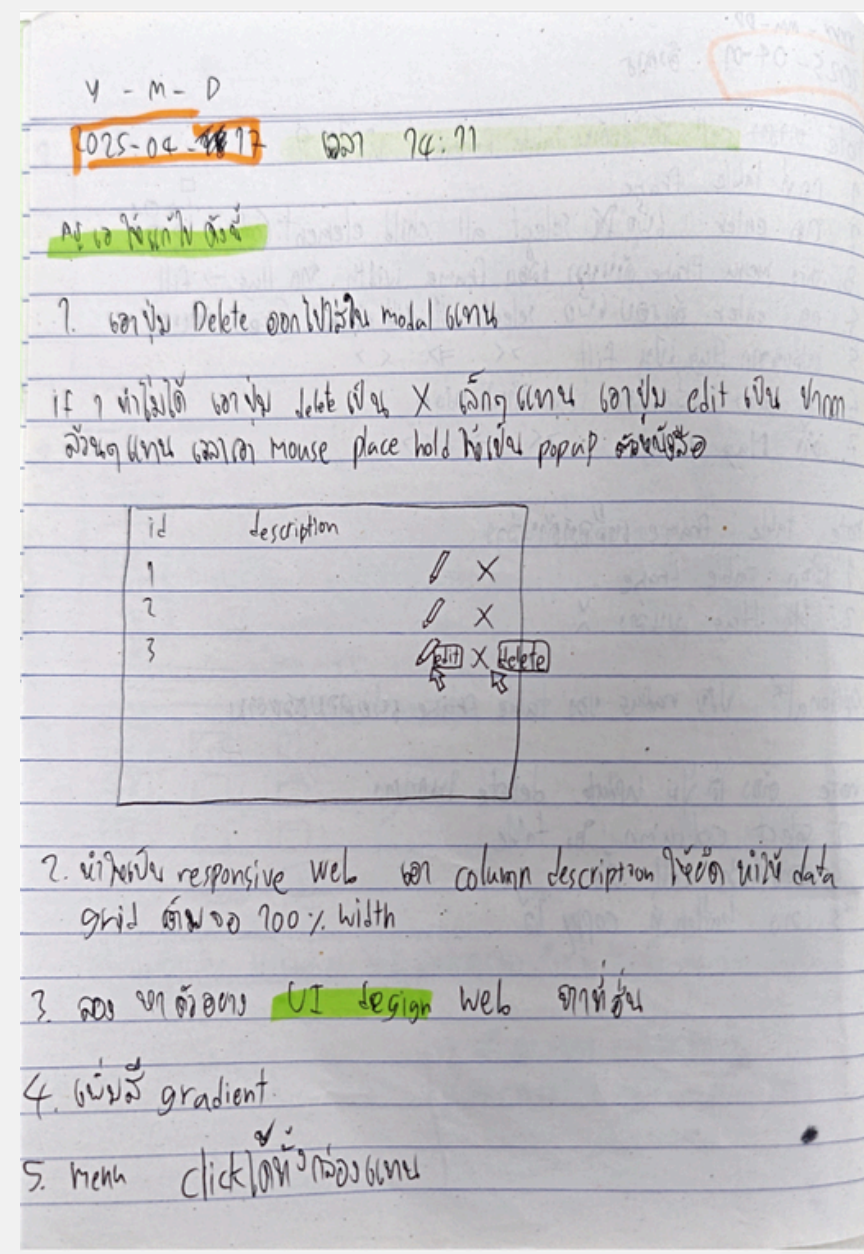
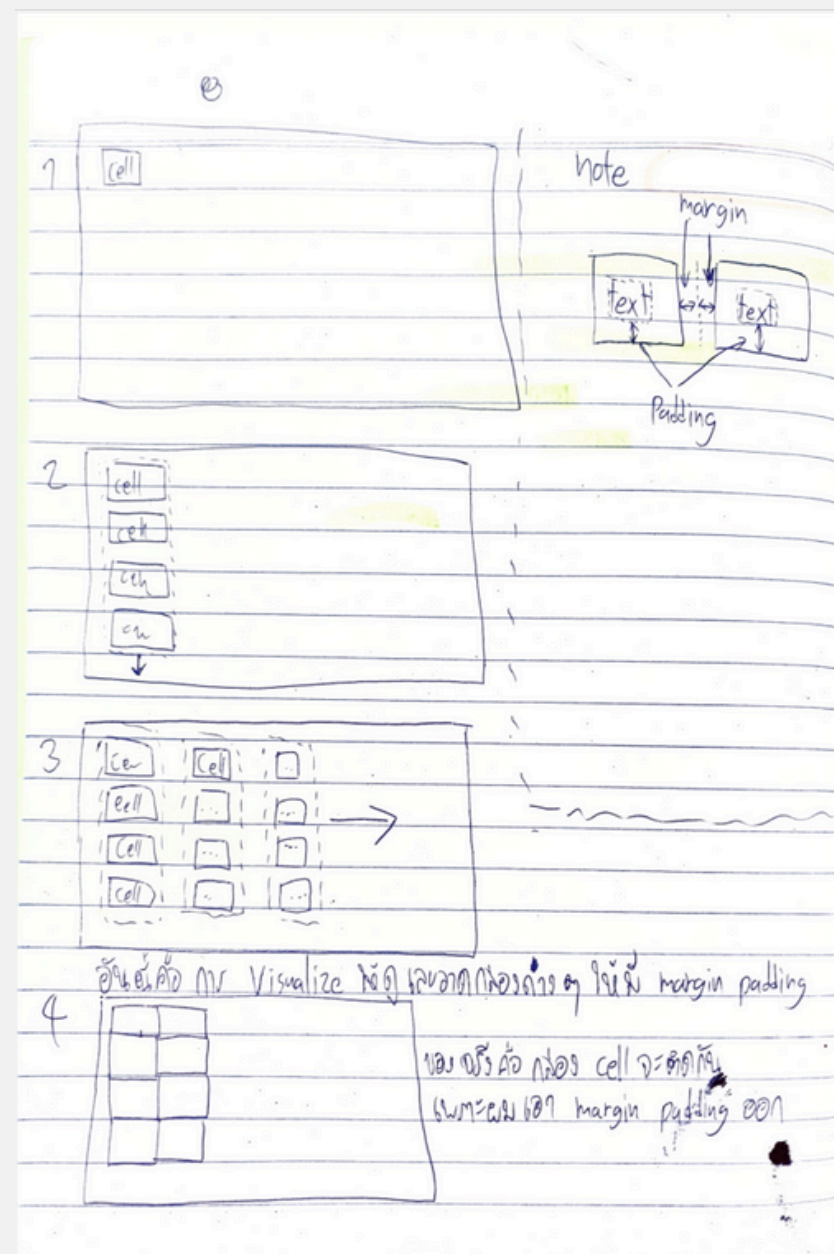
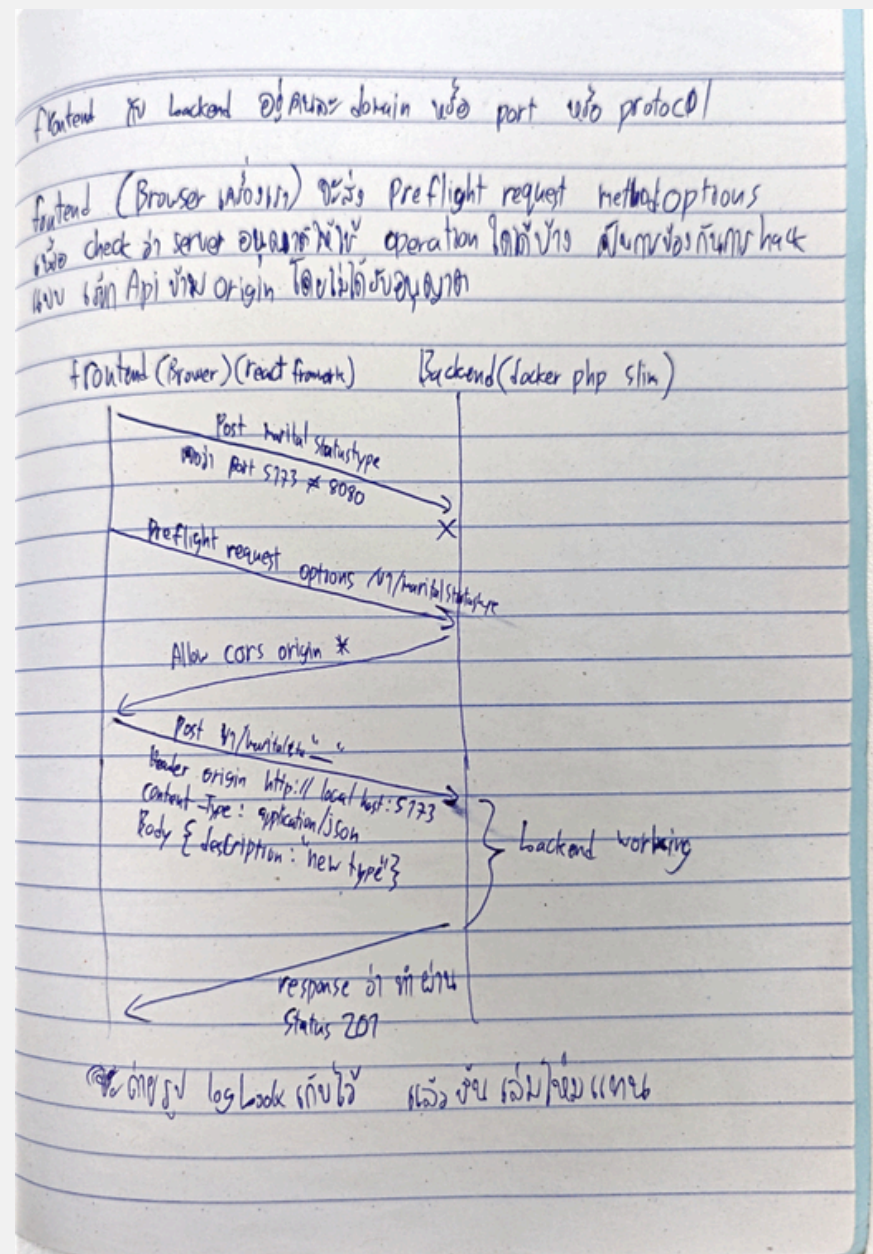


Project API and Web Page Summary

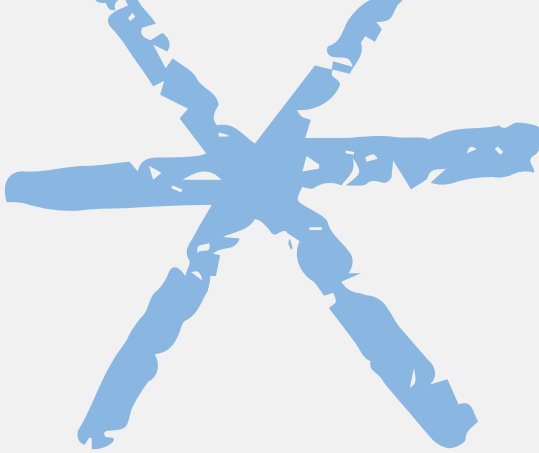
Project	API Services	Web Pages	Total
Latest Project	19	39	58
Geographic Boundary	15	17	32
Party Communication	43	54	97
Total	77	110	187



จำนวน API Service และ Web Page

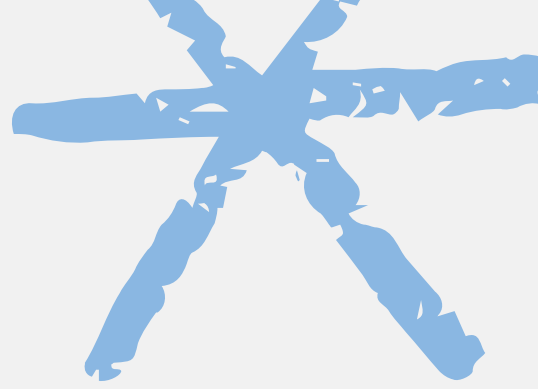


ตัวอย่าง Log Book รวมทั้งหมด 81 หน้า



ความท้าทายที่เจอ

- ออกแบบ Database ให้ต่อยอดง่ายและมีประสิทธิภาพยาก
 - สร้าง Full Stack Cloud Native App ซับซ้อน ต้องเรียนรู้แยกส่วน
 - Data Model Resource Book เก่า อ่านยาก เข้าใจลำบาก
 - JWT Authentication สำหรับ 6 บทบาทผู้ใช้อาจมีช่องโหว่
 - จัดการ hashed password ในระบบ login ซับซ้อน
 - ออกแบบ UX/UI ใช้งานง่ายสำหรับ Gen Z ทำทาย
 - จัดการข้อมูลวันเวลายุ่งยาก (time zone, date form)
 - เก็บประวัติข้อมูลว่าใครทำอะไร เมื่อไหร่ เป็นเรื่องซับซ้อน
- 



แนวทางต่อยอด

- พัฒนา Full Stack App ให้รองรับทุกภาษา
- ใช้ Reference Data Model อื่นๆ ที่ซับซ้อนและมีประสิทธิภาพมากกว่านี้
- UX/UI ตั้งค่าเปลี่ยน theme เว็บได้ตามต้องการ
- เพิ่ม AI/ML วิเคราะห์ Communication
- ขยายสู่ Mobile App (ใช้ React Native แบบ via bus)
- นำ App ที่ได้ ไป Deploy ในระบบ Cloud Computing อื่นๆ

Q

&

A